

Especificación de Requisitos de Software (SRS) — SGB-SaaS

Sistema de Gestión Bibliotecaria Web Formato basado en ISO/IEC/IEEE 29148:2018 (Systems and software engineering — Life cycle processes — Requirements engineering).

- **Proyecto:** SGB-SaaS, Proyecto Fin de Curso (PFC), asignatura Aplicaciones Web, UTEQ (2026-2027)
- **Equipo:** Loor Medranda Marlon Taylor (Tech Lead / DevOps / Seguridad), Cajas Ibarra Irvin Marcelo (Backend), Panama Murillo Moises Antonio (Frontend)
- **Versión:** v1.0.0 — Entrega Final. Versión anterior archivada en `docs/requisitos/historico/SRS-v0.9.0-rc.md` (estado de la Tercera Entrega, 30 requisitos; hash de commit base original invalidado por una reescritura posterior de historia con `git-filter-repo`, no resoluble en este repositorio).
- **Commit base de este documento:** ancla histórica invalidada por la misma reescritura de historia — esta revisión (2026-09-07) se ancla en cambio al tag v1.0.0 (commit 16279881), verificado presente y alcanzable en este repositorio.
- **Repositorio:** <https://github.com/mloorm14/sgb-saas>
- **Fuente de trazabilidad:** `docs/trazabilidad/matriz.csv` (71 requisitos tras esta revisión: 43 de la versión anterior + 3 ya existentes en la matriz sin redactar en el SRS (REQ-F-029/030/031, ahora redactados) + 20 nuevos (REQ-F-032-042, REQ-NF-016-024) + 5 filas netas adicionales de dividir REQ-NF-014 en 4 sub-IDs y REQ-F-022 en 3 = 43+3+20+5 = 71; validada automáticamente en CI por `scripts/validate-traceability.sh`, extendido en esta misma revisión con 2 validaciones nuevas — ver `CHANGELOG-REQ.md`)

Nota de método. Este documento **no redacta requisitos nuevos desde cero**: consolida y da estructura formal IEEE 29148 a lo que ya existía disperso en el repositorio antes de este commit — `docs/trazabilidad/matriz.csv` (43 requisitos con ID/tipo/prioridad/endpoint/prueba/evidencia), `docs/requisitos/historias/` y `docs/requisitos/casos-de-uso/` (formato Connextra + Cockburn), `docs/requisitos/historias-usuario.md` y `docs/requisitos/casos-de-uso.md` (los 5 HU/CU del módulo de Cajas que no siguen la convención de un archivo por HU), los ADRs de `docs/adr/` y el resumen ejecutivo de `docs/informe-entrega-3.tex`. Donde el repositorio no tenía evidencia real que respalde un rationale o un criterio de aceptación, este documento lo declara explícitamente en vez de inventarlo — ver la nota de honestidad de cada requisito afectado y el resumen en la sección 6.

Actualización a v1.0.0 (Entrega Final): esta versión agrega los 13 requisitos (REQ-F-017 a REQ-F-028, REQ-NF-015) que la matriz de trazabilidad ya documentaba pero que no tenían entrada correspondiente en el SRS — los 8 módulos construidos por Cajas después del commit base de la versión anterior (hash de commit invalidado por la misma reescritura de historia citada arriba; previo al merge de sus 8 ramas): verificación de correo, credencial QR, notificaciones, favoritos/ sugerencias de adquisición, panel de administración y auditoría, configuración paramétrica, reportes (morosidad/uso/PDF) y el chatbot con Gemini. Para la mayoría de estos módulos **no existe HU/CU dedicada en el repositorio** (la matriz cita IDs como HU-CFG-01/HU-ADM-01/ HU-AUD-01/HU-PRE-03/CU-CFG-01/CU-07/CU-ADM-01/CU-AUD-01 que no corresponden a ningún archivo real en `docs/requisitos/historias/` ni `docs/requisitos/casos-de-uso/` — verificado por búsqueda exhaustiva en el repositorio antes de escribir esta versión) — se declara como gap en cada requisito afectado y en el resumen de la sección 6, sin inventar el contenido de esas HU/CU. También se corrigió el estado de REQ-NF-012 y REQ-NF-014 (TLS/CSP), que la versión anterior documentaba como “pendiente” porque en ese momento lo estaban — ambos se cerraron parcialmente después, vía `feature/seguridad-transporte`, y la matriz ya lo refleja; ver el detalle en cada requisito.

Hallazgo identificado y ya resuelto dentro de esta misma revisión (Dr. Guerrero, 2026-09-07): en un punto intermedio de esta tarea de corrección, `docs/trazabilidad/matriz.csv` tenía 46 filas, no 43 — REQ-F-029, REQ-F-030 y REQ-F-031 existían en la matriz sin entrada correspondiente en la sección 3 de este documento. Quedaron identificados primero (para no perder el hallazgo) y redactados después, en la misma tarea, junto con 20 requisitos nuevos más — ver `docs/requisitos/CHANGELOG-REQ.md` para la cronología completa por bloque. La matriz y este documento quedan sincronizados al cierre de esta revisión: **71 filas, 0 problemas** en `scripts/validate-traceability.sh`.

Historial de revisiones

Fecha de emisión de esta revisión: 2026-09-07.

Esta tabla resume el historial a nivel de versión del documento; el detalle línea por línea de cada requisito

modificado/agregado/dividido en esta revisión vive en `docs/requisitos/CHANGELOG-REQ.md`, que esta tabla referencia en vez de duplicar.

Versión	Fecha	Requisitos	Cambio principal
v0.9.0-rc	Tercera Entrega (histórico)	30	Versión inicial, ver <code>historico/SRS-v0.9.0-rc.md</code>
v1.0.0 (Entrega Final, previo a esta revisión)	2026-08-12 (commit histórico, hash invalidado por <code>git-filter-repo</code>)	43	Agrega los 13 requisitos de los 8 módulos de Cajas (REQ-F-017-028, REQ-NF-015); corrige estado de REQ-NF-012/REQ-NF-014
v1.0.0 (esta revisión)	2026-09-07	71	Auditoría completa contra ISO/IEC/IEEE 29148:2018 (Dr. Gleiston Guerrero): corrige contradicciones internas (M1-M4), sincroniza cifras y estados con el código real (M5-M24), divide 2 requisitos compuestos (M14), agrega 25 requisitos nuevos para funcionalidad ya implementada sin especificar (A1-A25), y ajustes de forma/lenguaje vinculante (M13, M15, M25-M29) — ver <code>CHANGELOG-REQ.md</code> para el detalle completo por bloque

1. Introducción

1.1 Propósito

Este documento especifica de forma completa y verificable los requisitos funcionales y no funcionales del sistema SGB-SaaS, consolidando en un solo artefacto formal (SRS) la información que hasta esta entrega vivía correcta pero dispersa entre la matriz de trazabilidad, las historias de usuario, los casos de uso y los Architecture Decision Records (ADR). Su audiencia es el equipo de desarrollo (para verificar que la implementación actual cumple lo especificado), el evaluador del PFC (para juzgar completitud y trazabilidad), y cualquier integrante futuro del equipo que necesite entender qué se construyó y por qué, sin tener que reconstruir ese razonamiento leyendo el código o el historial de Git.

1.2 Alcance

El sistema especificado es **SGB-SaaS**: una plataforma web de gestión bibliotecaria para bibliotecas institucionales/municipales, con los módulos Auth (registro, login, logout, refresco de sesión, verificación de correo, control de acceso por rol), Libros/Catálogo (CRUD, favoritos, sugerencias de adquisición), Préstamos (creación, devolución, renovación, reportes), Reservaciones (creación, listado), Multas (listado, pago, anulación), Credencial QR (identificación del lector sin escribir su usuario), Notificaciones (alertas de vencimiento/multa/reserva caducada), Panel de administración (gestión de usuarios/roles) y auditoría, Configuración paramétrica del sistema, y un asistente virtual (Chatbot) con grounding real sobre el catálogo. El alcance de este SRS cubre exactamente los **71 requisitos** ya identificados y trazados en `docs/trazabilidad/matriz.csv` al momento de este commit (los 30 originales de la Tercera Entrega, más los 13 de los módulos construidos después, más los 28 que esta revisión agrega/formaliza — REQ-F-029-042, REQ-NF-016-024 y la división de REQ-NF-014/ REQ-F-022 (ambos IDs padre retirados — no se reutilizan, usar solo los sub-IDs — ver `CHANGELOG-REQ.md` para el detalle completo) — no se amplía el alcance funcional del sistema al redactar este documento (el sistema construido no cambia), solo se formaliza su especificación, que es precisamente el hallazgo central del Dr. Guerrero que motivó esta revisión. Explícitamente **fuera de alcance** de este documento (y del sistema, en esta entrega): integración con sistemas académicos institucionales externos, TLS gestionado por este propio repositorio (el sistema **sí corre bajo HTTPS real en producción** — Render termina TLS en su borde para `sgb-backend/biblora-sgb`, ver REQ-NF-012 — pero ningún certificado ni configuración `server.ssl.*` vive en este repositorio ni en el stack de Docker Compose local, que sigue siendo HTTP plano — verificado por ausencia de configuración TLS/443 en `docker-compose.yml` y `frontend-angular/nginx.conf`), integración con Google Books API (retirada del modelo C4 por no existir en el código, ver `docs/arquitectura/workspace.dsl`), y los sub-bloques de evidencia empírica de usabilidad (SUS) que dependen de participantes humanos reales, no automatizables — ver OBS-08 en `docs/observaciones/OBSERVACIONES.md`, todavía pendiente al momento de este commit. **Nota (hallazgo del Dr. Guerrero)**: este párrafo ya mencionaba favoritos y sugerencias de adquisición dentro de “Libros/Catálogo” sin que existiera un requisito REQ-F-XXX formal que los especificara — esta actualización cierra ese gap con A4 (favoritos) y A5 (sugerencias de adquisición), ver Bloque 5.

1.3 Definiciones, acrónimos y abreviaturas

Término	Significado
SRS	Software Requirements Specification (este documento)
HU / CU	Historia de Usuario / Caso de Uso
ADR	Architecture Decision Record
RBAC	Role-Based Access Control (control de acceso basado en roles)
JWT	JSON Web Token (RFC 7519)
SP	Stored Procedure (procedimiento o función almacenada en PostgreSQL)
ORM	Object-Relational Mapping (Spring Data JPA / Hibernate en este proyecto)
DTO	Data Transfer Object
TTL	Time To Live (tiempo de expiración de una entrada de cache o de una clave en Redis)
RLS	Row Level Security (PostgreSQL)
MoSCoW	Must / Should / Could / Won't — escala de priorización de requisitos
CRUD	Create, Read, Update, Delete
OWASP	Open Web Application Security Project (Top 10:2021 usado como marco de referencia de seguridad)
PFC	Proyecto Fin de Curso
UTEQ	Universidad Técnica Estatal de Quevedo
SQLSTATE	Código de error de 5 caracteres devuelto por PostgreSQL (LB404/LB409/LB422 son códigos custom de este proyecto, ver GlobalExceptionHandler)

1.3.1 Estados del dominio (catálogo cerrado y transiciones)

Conjunto **cerrado** de valores por entidad, verificado línea por línea contra las sentencias INSERT de `db/seed.sql` (no se asume ningún valor no sembrado). Los nombres son los que exigen los procedimientos/funciones SQL y el código Java por igual (`db/seed.sql`, líneas 9-13: “cualquier cambio aquí debe reflejarse también allá”).

Entidad	Estados (orden de inserción en db/seed.sql)
usuarios (estados_usuario)	ACTIVO, BLOQUEADO_POR_MULTA, INACTIVO, PENDIENTE_VERIFICACION
libros (estados_libro)	ACTIVO, DADO_DE_BAJA, EN_REPARACION, PERDIDO
prestamos (estados_prestamo)	ACTIVO, RENOVADO, DEVUELTO, VENCIDO
multas (estados_multa)	PENDIENTE, PAGADA, ANULADA
reservaciones (estados_reservacion)	PENDIENTE, LISTA_PARA_RETIRO, RETIRADA, EXPIRADA, CANCELADA

Tabla de transiciones (evento/actor real que dispara cada cambio, verificado en el código de los `*Service/*Scheduler` correspondientes; no existe un `UsuarioService` dedicado — las transiciones de `usuarios` viven repartidas entre `AuthService`, `VerificacionCorreoService`, `UsuarioAdminService` y los procedimientos SQL de multas):

Entidad	Transición	Disparador
usuario	(registro) → PENDIENTE_VERIFICACION	<code>AuthService.registrar</code> (POST <code>/api/auth/registro</code>)
usuario	PENDIENTE_VERIFICACION → ACTIVO	<code>AuthService.verificarCorreo</code> tras código correcto (POST <code>/api/auth/verificar-correo</code>)
usuario	ACTIVO → BLOQUEADO_POR_MULTA	<code>sp_registrar_devolucion</code> cuando la devolución genera multa por atraso
usuario	BLOQUEADO_POR_MULTA → ACTIVO	<code>sp_pagar_multa</code> , solo si era la última multa PENDIENTE del usuario
usuario	ACTIVO ↔ INACTIVO	<code>UsuarioAdminService.cambiarEstado</code> (ADMIN sin restricción de conjunto; GERENTE restringido a ACTIVO/INACTIVO y solo sobre usuarios que él mismo creó)
usuario	cualquiera → INACTIVO	<code>UsuarioAdminService.eliminarUsuario</code> (baja lógica, DELETE <code>/api/v1/admin/usuarios/{id}</code>)
libro	(creación) → ACTIVO	<code>LibroService.crear</code> , salvo la excepción de la fila siguiente
libro	ACTIVO → DADO_DE_BAJA	<code>LibroService.eliminar</code> (baja lógica, nunca borrado físico)
libro	ACTIVO/DADO_DE_BAJA/EN_REPARACION/PERDIDO (cualquier transición manual)	<code>LibroService.actualizar</code> , campo <code>estadoId</code> del request — quien edita elige el estado del catálogo directamente; ningún flujo automático transiciona a EN_REPARACION/PERDIDO (ver nota de honestidad abajo)
préstamo	(creación) → ACTIVO	<code>PrestamoService.crear</code> (<code>sp_crear_prestamo</code>)
préstamo	ACTIVO/RENOVADO → DEVUELTO	<code>sp_registrar_devolucion</code> (POST <code>/api/v1/prestamos/{id}/devolucion</code>)
préstamo	ACTIVO → RENOVADO	<code>PrestamoService.renovar</code> (POST <code>/api/v1/prestamos/{id}/renovacion</code>)
multa	(generación por atraso) → PENDIENTE	<code>sp_registrar_devolucion</code>
multa	PENDIENTE → PAGADA	<code>sp_pagar_multa</code> (POST <code>/api/v1/multas/{id}/pago</code>)

Entidad	Transición	Disparador
multa	PENDIENTE → ANULADA	sp_anular_multa (POST /api/v1/multas/{id}/anulacion, solo GERENTE/ADMIN)
reservación	(creación) → PENDIENTE	ReservacionService.crear (POST /api/v1/reservaciones)
reservación	PENDIENTE → LISTA_PARA_RETIRO	ReservacionService (aceptación del staff, PATCH de cambio de estado)
reservación	PENDIENTE → CANCELADA	ReservacionService (rechazo del staff, mismo endpoint)
reservación	PENDIENTE/LISTA_PARA_RETIRO → RETIRADA	PrestamoService.crear cuando el préstamo se vincula a una reservacionId
reservación	PENDIENTE/LISTA_PARA_RETIRO → EXPIRADA	ReservacionScheduler.expirarReservacionesVencidas (job cada 15 min, spExpirarReservacionesVencidas)

Notas de honestidad (verificadas en este commit, no asumidas):

1. **VENCIDO (estados_prestamo) nunca se asigna en el código.** Es un estado sembrado en el catálogo, pero “vencido” se calcula **dinámicamente** comparando `fechaDevolucionEstimada` contra `OffsetDateTime.now()` en el momento de la operación (ej. `PrestamoService.renovar`, línea `if (prestamo.getFechaDevolucionEstimada().isBefore(OffsetDateTime.now()))` — ningún UPDATE ni procedimiento persiste `estado_prestamo_id` como VENCIDO. Un préstamo atrasado sigue mostrando ACTIVO/RENOVADO en la columna de estado hasta que se devuelve.
2. **EN_REPARACION/PERDIDO (estados_libro) no tienen transición automática.** El flujo de registro de daños al devolver un préstamo (`DevolucionService.registrarDevolucion`, con `dto.estadoDevolucion()` en `{CON_DANO, PERDIDO}`) crea un `RegistroDano` y, si aplica, una multa adicional por daño — pero **no modifica** el `estado_libro_id` del libro afectado. Los dos estados solo son alcanzables editando el libro manualmente (PUT /api/v1/libros/{id}, campo `estadoId`).
3. **Referencia a un estado PENDIENTE de estados_libro que no existe en el seed.** `LibroService.crear` busca `estadoRepo.findByNombre("PENDIENTE")` (vía `.orElse(null)`, sin lanzar excepción) para un flujo de “revisión pendiente” al crear un libro como GERENTE/ADMIN; `db/seed.sql` **no siembra ninguna fila PENDIENTE en estados_libro** (solo ACTIVO/DADO_DE_BAJA/ EN_REPARACION/PERDIDO), así que esa búsqueda siempre devuelve vacío y la rama `if (pendiente != null)` nunca se ejecuta contra los datos sembrados de este repositorio. Un comentario en el mismo archivo (`LibroService.java`, cerca de la línea 187) asume una numeración de IDs 2,3,4,5 para DADO_DE_BAJA,PENDIENTE,EN_REPARACION,PERDIDO, que tampoco coincide con el orden real de 4 filas sembradas (ACTIVO=1, DADO_DE_BAJA=2, EN_REPARACION=3, PERDIDO=4, sin id 5). `PENDIENTE_VERIFICAR_MARLON`: confirmar si `estados_libro` debería tener una quinta fila PENDIENTE (y agregarla a `db/seed.sql`) o si ese código es vestigial de un diseño descartado.

Ver también A24 (sección 3.1, Bloque 5 de esta actualización), que referencia esta misma tabla como anexo formal de trazabilidad.

1.4 Referencias

- ISO/IEC/IEEE 29148:2018 — Requirements Engineering (estructura de este documento).
- ISO/IEC 25010:2011 — Systems and software Quality Requirements and Evaluation (SQuaRE), aplicado en `docs/arquitectura/ISO25010.md`.
- OWASP Top 10:2021.
- RFC 7519 (JSON Web Token), RFC 7807 (Problem Details for HTTP APIs).
- `docs/trazabilidad/matriz.csv` — fuente primaria de los 71 requisitos.
- `docs/requisitos/historias/`, `docs/requisitos/casos-de-uso/`, `docs/requisitos/historias-usuario.md`, `docs/requisitos/casos-de-uso.md`.
- `docs/adr/ADR-001-tecnologia.md`, `ADR-003-jwt-redis.md`, `adr-006 a adr-016`, `adr-029-v29-gap.md` (14 ADRs — cifra recontada el 2026-09-07 directamente sobre `docs/adr/`, excluyendo `README.md`; corrige la cifra de 13 de versiones anteriores de este SRS, desactualizada por la incorporación posterior de `adr-029`).
- `docs/informe-entrega-3.tex` (resumen ejecutivo, estado del sistema, inventario de endpoints).
- `docs/arquitectura/ISO25010.md`, `docs/arquitectura/workspace.dsl` (C4).
- `docs/basedatos/CATALOGO-SP.md` (catálogo de los 7 procedimientos/funciones SQL).

1.5 Resumen del documento

La sección 2 describe el producto de forma global (perspectiva, funciones, usuarios, restricciones, supuestos). La sección 3 es el cuerpo principal: los 71 requisitos específicos, cada uno con id único, descripción, rationale, prioridad MoSCoW,

criterio de aceptación medible y método de verificación. La sección 4 resume el mecanismo de trazabilidad hacia código/pruebas/evidencia. La sección 5 mapea los requisitos no funcionales contra ISO/IEC 25010. La sección 6 declara explícitamente los gaps y limitaciones honestas encontradas al consolidar este documento.

1.6 Correspondencia con el Anexo C de ISO/IEC/IEEE 29148:2018

Agregada en esta revisión (M25, hallazgo del Dr. Guerrero). Este documento sigue la estructura clásica **IEEE 830** (Introducción / Descripción general / Requisitos específicos / Trazabilidad / Anexos), heredada de versiones anteriores del SRS y ya aceptada explícitamente por el docente como válida en vez de exigir una reestructuración completa a la plantilla informativa de SRS del Anexo C de 29148:2018 — **no se reordena el documento** en esta revisión, solo se deja constancia de la correspondencia y se justifica cada desviación.

Nota de honestidad sobre el alcance de esta tabla: este repositorio no tiene una copia versionada del texto completo de ISO/IEC/IEEE 29148:2018 (es un estándar con licencia, no de acceso libre) contra la cual verificar la numeración exacta de cláusulas del Anexo C carácter por carácter. La correspondencia de abajo usa la estructura de la plantilla informativa de SRS de ese anexo tal como es de conocimiento público y ya la cita la sección 1 de este propio documento (“Formato basado en ISO/IEC/IEEE 29148:2018”) — a nivel de categorías de contenido, no de número de cláusula certificado.

Sección de este SRS	Categoría equivalente, Anexo C (SRS) de 29148:2018	Desviación / justificación
1. Introducción (1.1-1.6)	Introduction (Purpose, Scope, Definitions, References, Overview)	Sin desviación de fondo — mismo contenido, con 1.6 (esta sección) y 1.3.1 (catálogo de estados, M12) como extensiones propias de este proyecto, no parte del template.
2. Descripción global	System/Product overview (perspectiva, funciones, usuarios, restricciones, supuestos)	Sin desviación de fondo.
3. Requisitos específicos (3.1-3.4)	Specific requirements (funcionales, interfaces, no funcionales)	Desviación real: 29148 no exige un formato de cuerpo por requisito específico; este documento usa un formato propio (Descripción + Rationale + Criterio de aceptación + Método de verificación), documentado y justificado en la sección 3.0, más cercano a un híbrido Volere/Cockburn/Gherkin que a una cláusula EARS de una sola oración — ver docs/checklists/incose2023-req.md , nota sistémica [S], para el análisis completo de esta desviación desde la óptica INCOSE (no redactado aquí para no duplicarlo).
3.4 Anexos (A23 matriz de permisos, A24 referencia a estados)	Podría vivir en Appendices	Se mantiene dentro de la sección 3 por cercanía temática con los requisitos que referencia (RBAC, catálogo de estados), en vez de moverlo al final del documento — decisión de legibilidad, no de conformidad.
4. Trazabilidad	No tiene equivalente directo en el template informativo de Anexo C (29148 trata trazabilidad como un proceso transversal, no una sección fija del SRS)	Sección propia de este proyecto, exigida por la guía del PFC (matriz de trazabilidad obligatoria), no por el estándar.
5. ISO/IEC 25010	Fuera del alcance del Anexo C de 29148 (25010 es un estándar de calidad de producto distinto, con su propio documento en este repositorio)	Sección propia, cross-referencia deliberada entre dos estándares distintos usados en este proyecto.
6. Notas de honestidad y gaps	No tiene equivalente en el template — es una práctica de este proyecto, no del estándar	Agregada por criterio propio del equipo para no dejar gaps implícitos; ver también docs/checklists/incose2023-req.md para el análisis de conformidad complementario.

2. Descripción global

2.1 Perspectiva del producto

SGB-SaaS es un sistema nuevo (no un reemplazo ni una migración de un sistema legado), construido como PFC con una arquitectura de tres capas: frontend SPA en Angular 21, backend API REST en Spring Boot 4.0.6 (Java 21), persistencia en PostgreSQL 16, y una capa de caché/blacklist de tokens en Redis 7. El sistema modela el ciclo completo de una biblioteca institucional: catálogo de libros, préstamos, reservaciones, multas y administración de usuarios bajo RBAC. No depende de ningún sistema externo para operar (no hay integraciones con sistemas académicos institucionales ni pasarelas de pago en el alcance actual). Los cuatro servicios (frontend, backend, PostgreSQL, Redis) se orquestan con Docker Compose (ADR-007) y se comunican dentro de una red Docker interna; el único punto de entrada externo es el frontend (puerto 4200) y, para pruebas directas/Swagger, el backend (puerto 8080).

2.2 Funciones del producto

A alto nivel (el detalle completo está en la sección 3):

- **Auth:** registro de cuentas, verificación de correo con código de un solo uso antes de poder iniciar sesión, login con emisión de JWT, logout con revocación inmediata, refresco de sesión vía cookie `HttpOnly`, bloqueo temporal tras intentos fallidos, auditoría de eventos de autenticación, control de acceso por rol en cada endpoint.
- **Libros/Catálogo:** consulta paginada del catálogo, alta/edición/baja lógica de libros, favoritos por usuario, sugerencias de adquisición con flujo de revisión (`FavoritoController`, `SugerenciaAdquisicionController` — funcionalidad real sin requisito formal propio hasta esta actualización; ver A4 y A5).
- **Préstamos:** creación (con validación de stock y estado del usuario), registro de devolución (con detección automática de atraso y generación de multa), renovación (con límite configurable de renovaciones y bloqueo si hay reserva vigente de otro usuario), listado de préstamos propios, reportes (libros más prestados, índice de morosidad, uso por período, exportación a PDF).
- **Reservaciones:** creación (a nombre propio si es LECTOR, a nombre de otro si es BIBLIOTECARIO/GERENTE), listado por usuario, expiración automática de reservas vencidas (job periódico).
- **Multas:** listado por usuario, pago (con desbloqueo condicional del usuario), anulación (restringida a GERENTE/ADMIN, con auditoría).
- **Credencial QR:** cada LECTOR puede consultar su propio código QR (identificación alternativa al usuario/contraseña para registrar un préstamo en el mostrador).
- **Notificaciones:** alertas automáticas de préstamo por vencer, multa generada y reserva caducada, con envío por correo (SMTP) y consulta desde la interfaz.
- **Panel de administración y auditoría:** gestión de rol/estado de cuentas de usuario (ADMIN), listado del padrón de usuarios (ADMIN y GERENTE), consulta de la bitácora de auditoría (GERENTE y ADMIN).
- **Configuración paramétrica:** parámetros del sistema (ej. máximo de renovaciones de un préstamo) editables en runtime por ADMIN, sin requerir un despliegue nuevo.
- **Chatbot (asistente virtual):** un LECTOR puede conversar con un asistente que responde con datos reales del catálogo/reservas (grounding), respaldado por Gemini 3.5 Flash Lite, con límite de mensajes por usuario.

2.3 Características de los usuarios

El sistema define 4 roles (modelo RBAC normalizado, ver ADR-010 y REQ-NF-010):

Rol	Perfil de usuario típico	Conocimiento técnico esperado
LECTOR	Estudiante o miembro de la comunidad universitaria que consulta el catálogo, pide préstamos/reservaciones y ve sus propias multas.	Ninguno — usuario final de una aplicación web convencional.
BIBLIOTECARIO	Personal de mostrador que registra préstamos, devoluciones y pagos de multas presencialmente.	Bajo — debe poder operar el sistema sin soporte técnico, la guía de usabilidad (ISO 25010, “Usabilidad: Alta”) asume esto explícitamente.
GERENTE	Responsable de la biblioteca; además de las funciones de BIBLIOTECARIO, puede anular multas y ver reportes.	Bajo-medio.
ADMIN	Administrador técnico/institucional del sistema; gestiona el catálogo (con la asimetría documentada en REQ-NF-010) y tiene visibilidad de auditoría.	Medio — se asume familiaridad con el dominio bibliotecario, no necesariamente con el sistema técnico subyacente.

2.4 Restricciones

- **Tecnológicas** (no negociables para esta entrega, ya decididas vía ADR): Spring Boot 4.0.6/Java 21 (ADR-001), Angular 21 (ADR-001), PostgreSQL 16 (ADR-011), Redis 7 (ADR-003/ADR-008), Docker Compose (ADR-007), Flyway 9 + `db/schema.sql/db/seed.sql` (ADR-013). Estrategia híbrida de acceso a datos obligatoria: CRUD elemental vía Spring Data JPA, operaciones multi-tabla vía procedimientos/funciones SQL (ADR-006, requisito explícito de la guía del PFC, Bloque A.2).
- **De despliegue:** el sistema debe poder levantarse completo con un solo comando (`make up / docker compose up --build`) contra un volumen de datos vacío (ADR-013, ADR-007) — requisito explícito del Bloque B de la guía (reproducibilidad).
- **De licenciamiento:** licencia MIT (ADR-009), requerida para la publicación del repositorio con DOI en Zenodo.
- **De entorno:** `JWT_SECRET` de mínimo 256 bits provisto vía `.env` (nunca hardcodeado ni committeado); ningún secreto vive en `docker-compose.yml` (ADR-007).

- **De tiempo del equipo:** 3 integrantes, sin dedicación exclusiva (proyecto académico) — restricción real que explica por qué ciertos requisitos quedan con estado “pendiente” (ver sección 6) en vez de simularse o fabricarse como completos.

2.5 Supuestos y dependencias

- Se asume que el evaluador/usuario final dispone de Docker y Docker Compose instalados (única dependencia dura del entorno de ejecución, ver README).
- Se asume disponibilidad de Redis para que el mecanismo de revocación de tokens (REQ-NF-001) y rate limiting (REQ-NF-006) funcionen. **Corrección (hallazgo del Dr. Guerrero, verificado leyendo JwtAuthFilter.java directamente):** este supuesto afirmaba en versiones anteriores de este SRS que, si Redis cae, JwtAuthFilter “queda sin forma de verificar revocaciones” — una política **fail-open** implícita. El código real hace exactamente lo contrario: JwtAuthFilter.doFilterInternal captura DataAccessException al consultar la blacklist y responde 401 explícitamente (SecurityContextHolder.clearContext() + 401 + ProblemDetail escrito a mano), es decir, **fail-closed** — ninguna request pasa sin poder confirmar la revocación. Este comportamiento se formaliza como requisito nuevo, ver A15 (Bloque 5 de esta actualización) para el detalle completo, incluida la comparación con los otros tres puntos de este sistema que sí dependen de Redis (LoginRateLimiter/ChatbotRateLimiter, fail-open; VerificacionCorreoService, fail-closed) — no todos se comportan igual, y A15 lo declara servicio por servicio en vez de asumir una política uniforme.
- Se asume un volumen de uso de biblioteca universitaria (bajo, no concurrencia tipo e-commerce) como base para las decisiones de rendimiento — ver REQ-NF-003 y la característica “Eficiencia de desempeño” (prioridad Media) de docs/arquitectura/ISO25010.md.
- Este documento depende de que docs/trazabilidad/matriz.csv siga siendo la fuente de verdad para IDs de requisitos; si la matriz cambia (se agregan/eliminan requisitos) sin actualizar este SRS, ambos documentos se desincronizan — mismo riesgo ya documentado en ADR-013 para el par Flyway/schema.sql.

3. Requisitos específicos

3.0 Convenciones usadas en cada requisito

Cada requisito **Must** incluye: **id único** (igual al de docs/trazabilidad/matriz.csv, para trazabilidad directa), **descripción**, **rationale** (por qué existe — basado en HU/CU/ADR reales, nunca inventado), **prioridad MoSCoW**, **criterio de aceptación medible** (derivado de los escenarios Gherkin reales de la HU/CU correspondiente cuando existen) y **método de verificación**: *Test* (prueba automatizada existente y en verde), *Demonstration* (verificado en vivo contra el stack real, con evidencia en docs/mediciones/), *Analysis* (decisión arquitectónica revisada por inspección/razonamiento, sin prueba automatizada de regresión), o *Inspection* (revisión directa del código fuente). Los requisitos **Should** se documentan con el mismo formato pero con menor exhaustividad cuando la fuente original (matriz/ADR) ya era menos detallada — no se rellena con contenido inventado para emparejar el formato.

Convención de lenguaje vinculante (agregada en esta revisión, hallazgo del Dr. Guerrero, M13): en el enunciado de todo requisito, “**debe**” es vinculante — el sistema tiene que cumplirlo para que el requisito se considere satisfecho. “**Debería**” no se usa en el enunciado de ningún requisito de este documento: donde antes aparecía (REQ-NF-012, REQ-NF-014a-d), se reescribió a “debe”; el matiz de cumplimiento parcial que “debería” insinuaba se expresa ahora en el campo explícito **Estado** (M28) y en la narrativa de “Estado real” de cada requisito afectado, no en el verbo del enunciado. Del mismo modo, REQ-NF-008 y REQ-NF-009 — que describían una decisión ya tomada en presente indicativo (“el sistema usa...”) en vez de como requisito vinculante — se reescribieron a forma “debe”.

3.1 Requisitos funcionales

REQ-F-001 — Registro de nuevo usuario

- **Prioridad:** Must
- **Estado:** verificado
- **Fuente:** HU-AUTH-01, CU-AUTH-01
- **Depende de:** REQ-F-020 (el estado inicial tras el registro solo existe porque REQ-F-020 introdujo el flujo obligatorio de verificación de correo; sin REQ-F-020 el estado inicial sería **ACTIVO** directo).
- **Módulo/endpoint:** AuthController/AuthService — POST /api/auth/registro

- **Descripción:** el sistema debe permitir que un visitante sin cuenta se registre con nombre, apellido, correo institucional y contraseña, quedando con rol `LECTOR` y estado `PENDIENTE_VERIFICACION` por defecto (conforme a REQ-F-020: no puede iniciar sesión hasta verificar el código enviado por correo).
- **Rationale:** sin registro propio, cualquier acceso al sistema dependería de que un administrador cree cada cuenta manualmente, lo cual no escala para una comunidad universitaria (HU-AUTH-01).
- **Criterio de aceptación medible:**
 1. Con correo no registrado y contraseña ≥ 8 caracteres, el sistema responde 201 con el usuario creado, rol `LECTOR`, estado `PENDIENTE_VERIFICACION` conforme a REQ-F-020, y la contraseña almacenada hasheada (nunca en texto plano).
 2. Con un correo ya registrado, el sistema responde 409 y no crea ningún usuario nuevo.
 3. Con una contraseña de menos de 8 caracteres, el sistema responde 400.
- **Método de verificación:** `AuthServiceTest.registroCorreoDuplicado` cubre el criterio 2 (rechazo por correo duplicado); `AuthServiceTest.registroExitoso_dejaAlUsuarioPendienteDeVerificacionYEnviaElC` (compartido con REQ-F-020) cubre la parte de estado `PENDIENTE_VERIFICACION` del criterio 1. **Nota de honestidad:** el resto del criterio 1 (código 201, contraseña hasheada) y el criterio 3 (rechazo por contraseña corta) **no tienen prueba automatizada de regresión propia** en este repositorio a la fecha de este documento; se documentan como parte del comportamiento especificado (visible en el Gherkin de HU-AUTH-01) pero no como verificados por test.

REQ-F-002 — Inicio de sesión

- **Prioridad:** Must
- **Estado:** verificado
- **Fuente:** HU-AUTH-02, CU-AUTH-02
- **Módulo/endpoint:** `AuthController/AuthService` — `POST /api/auth/login`
- **Descripción:** el sistema debe autenticar a un usuario registrado con correo y contraseña, emitiendo un `accessToken` en el cuerpo y un `refreshToken` en cookie `HttpOnly`.
- **Rationale:** es el punto de entrada de todo el control de acceso RBAC del resto del sistema (HU-AUTH-02, ADR-010).
- **Criterio de aceptación medible:**
 1. Credenciales correctas + usuario `ACTIVO` → 200 con `accessToken` y cookie `refreshToken` (`HttpOnly`, `Secure`, `SameSite=Strict`).
 2. Contraseña incorrecta → 401, sin emitir ningún token.
 3. Usuario `BLOQUEADO_POR_MULTA` → 423.
 4. Usuario `INACTIVO/PENDIENTE_VERIFICACION` → 403.
- **Método de verificación:** `Test` (`AuthServiceTest.login*`, 5 tests)
 - **Demonstration** (`docs/mediciones/sec/owasp/2026-07-30-owasp-a07-fix-rate-limiting-login.md`, `docs/mediciones/sec/owasp/2026-07-30-owasp-a09-fix-logging-autenticacion.md` — verificación en vivo contra el stack Docker real).

REQ-F-003 — Cierre de sesión

- **Prioridad:** Must
- **Estado:** verificado
- **Fuente:** HU-AUTH-03, CU-AUTH-03
- **Módulo/endpoint:** `AuthController/AuthService` — `POST /api/auth/logout`
- **Descripción:** el sistema debe invalidar de inmediato el `accessToken` de la sesión activa al cerrar sesión, aunque no haya expirado aún.
- **Rationale:** reduce la ventana de riesgo si el dispositivo queda desatendido o el token fue comprometido (HU-AUTH-03, ADR-003).
- **Criterio de aceptación medible:**
 1. Logout responde 204.
 2. El `accessToken` usado queda en blacklist (Redis) hasta su expiración natural.
 3. Cualquier request posterior con ese mismo token es rechazado.
 4. La cookie `refreshToken` se limpia (`maxAge=0`).
 5. El evento `LOGOUT` queda registrado (correo, IP, fecha/hora).
- **Método de verificación:** `Test` (`AuthServiceTest.logoutGuardaTokenEnBlacklist`) + **Demonstration** (`docs/mediciones/sec/owasp/2026-07-30-owasp-a09-fix-logging-autenticacion.md`).

REQ-F-004 — Refresco de sesión

- **Prioridad:** Must
- **Estado:** verificado
- **Fuente:** HU-AUTH-04, CU-AUTH-04
- **Módulo/endpoint:** AuthController/AuthService — POST /api/auth/refresh
- **Descripción:** el sistema debe emitir un `accessToken` nuevo a partir de una cookie `refreshToken` válida, sin exigir que el usuario vuelva a escribir su contraseña.
- **Rationale:** evita interrupciones de sesión cada hora (vida del `accessToken`) sin comprometer el `refreshToken` (que nunca es legible por JavaScript, ver ADR-012) — HU-AUTH-04.
- **Criterio de aceptación medible:**
 1. Cookie `refreshToken` válida presente → 200 con `accessToken` nuevo.
 2. Sin cookie `refreshToken` → 400.
 3. `refreshToken` inválido o expirado → no se emite token nuevo, responde 401, no 500 (ver evidencia empírica en docs/trazabilidad/matriz.csv para el detalle del fix que corrigió este código de estado — hallazgo del Dr. Guerrero: el hash de commit que documentaba este fix quedó invalidado por una reescritura posterior de historia con `git-filter-repo`; se traslada al campo `evidencia_empirica` de la matriz, anclado al tag v1.0.0, commit 16279881, en vez del criterio de aceptación).
- **Método de verificación:** Test (AuthServiceTest.refreshConTokenValido) + **Demonstration** (docs/mediciones/sec/

REQ-F-005 — Consultar el catálogo de libros

- **Prioridad:** Must
- **Estado:** verificado
- **Fuente:** HU-LIB-01, CU-LIB-01
- **Módulo/endpoint:** LibroController/LibroService — GET /api/v1/libros, GET /api/v1/libros/{id}
- **Descripción:** cualquier usuario autenticado (LECTOR o superior) debe poder ver el listado paginado del catálogo y el detalle de un libro. **Alcance acotado (hallazgo del Dr. Guerrero):** este requisito aplica únicamente al catálogo **autenticado** (GET /api/v1/libros); existe un segundo camino de consulta del catálogo **sin autenticación**, bajo /api/publico/** (PublicoLibroController, PublicoCategoriaController, permitAll en SecurityConfig.java), que es un requisito distinto — ver A6 para el portal público sin cuenta.
- **Rationale:** es la operación de lectura más frecuente del sistema (HU-LIB-01, docs/arquitectura/IS025010.md — “Eficiencia de desempeño”), de ahí también su cache Redis (REQ-NF-003).
- **Criterio de aceptación medible:**
 1. Listado paginado → 200, ordenado por título, con stock disponible por libro.
 2. Detalle de libro existente y ACTIVO → 200 con datos completos.
 3. Libro inexistente o DADO_DE_BAJA → 404.
- **Método de verificación:** Test (LibroServiceTest.listar_retornaPaginaDeLibros, .buscarPorId_cuandoNoExiste_libro, LibroControllerSecurityTest, 4 tests con @PreAuthorize real vía MockMvc) + **Demonstration** (docs/mediciones/sec/owasp/2026-07-30-owasp-a01-fix-rol-admin-libros.md).

REQ-F-006 — Gestionar el catálogo de libros

- **Prioridad:** Must
- **Estado:** verificado
- **Fuente:** HU-LIB-02, CU-LIB-02
- **Módulo/endpoint:** LibroController/LibroService — POST/PUT/DELETE /api/v1/libros{/,/id}
- **Descripción:** BIBLIOTECARIO/GERENTE/ADMIN deben poder crear, editar y dar de baja (lógicamente, nunca borrado físico) libros del catálogo.
- **Rationale:** mantiene el catálogo actualizado con los ejemplares reales de la biblioteca (HU-LIB-02).
- **Criterio de aceptación medible:**
 1. ISBN nuevo + datos válidos → 201.
 2. ISBN duplicado → 400.
 3. Edición de libro existente → 200 con datos actualizados.
 4. Baja de libro ACTIVO → 204, pasa a DADO_DE_BAJA (fila preservada, no borrada), deja de aparecer en listado/detalle.
 5. Rol LECTOR intentando gestionar → 403.
- **Método de verificación:** Test (LibroServiceTest.crearLibro_cuandoIsbnNuevo_retornaDT0, .crearLibro_cuandoIsbnDuplicado_retorna400, .eliminar_cuandoExiste_loMarcaDadoDeBaja, LibroControllerSecurityTest, 4 tests) + **Demonstration**

REQ-F-007 — Registrar préstamo

- **Prioridad:** Must
- **Estado:** verificado
- **Fuente:** HU-01 (Cajas, en docs/requisitos/historias-usuario.md), CU-01 (docs/requisitos/casos-de-uso.md)
- **Módulo/endpoint:** PrestamoController/PrestamoService — POST /api/v1/prestamos (SP sp_crear_prestamo)
- **Descripción:** un BIBLIOTECARIO/GERENTE debe poder registrar el préstamo de un libro con stock disponible a un usuario ACTIVO, decrementando el stock en la misma transacción atómica.
- **Corrección de rol — RESUELTA (hallazgo del Dr. Guerrero, verificado en código al construir A23 — matriz de permisos; corregida el 2026-09-07, ver OBS-27):** PrestamoController.crear (POST /api/v1/prestamos) tenía @PreAuthorize("hasAnyRole('GERENTE','ADMIN')") — BIBLIOTECARIO no estaba en la lista, pese a que esta misma descripción y REQ-F-016 (la UI de gestión de préstamos) asumen que un BIBLIOTECARIO puede registrar préstamos. Era una contradicción real entre lo documentado (incluido en versiones anteriores de este SRS) y el código, no una corrección de redacción menor — un usuario únicamente BIBLIOTECARIO recibía 403 al intentar crear un préstamo. Confirmado con Marlon (tech lead) que se trataba de un bug y no de una decisión de diseño; el @PreAuthorize se corrigió a hasAnyRole('BIBLIOTECARIO','GERENTE','ADMIN') y quedó verificado con tests reales (PrestamoControllerSecurityTest.crear_conRolBibliotecario_sePermite, más crear_conRolLector_seRechaza confirmando que el resto de roles sin permiso sigue recibiendo 403) — ver A23 para el detalle completo de la matriz de permisos y la comparación con el resto de endpoints del módulo.
- **Rationale:** núcleo del dominio bibliotecario — llevar control de qué ejemplares están fuera y cuándo deben devolverse (HU-01). La atomicidad de “crear préstamo + decrementar stock” está garantizada por el motor (sp_crear_prestamo), no por disciplina de código Java (ADR-006).
- **Criterio de aceptación medible:**
 1. Libro con stock > 0 y usuario ACTIVO → préstamo ACTIVO, stock decrementado en 1.
 2. Libro sin stock → 422 (“sin stock disponible”), sin crear registro.
 3. Usuario BLOQUEADO_POR_MULTA → 422 (“multas pendientes”), sin crear registro.
 4. Usuario o libro inexistente → 404.
 5. El campo diasPrestamo del request es **obligatorio** (@NotNull) y debe ser un entero >=1 (@Min(1), PrestamoRequestDTO.java); no hay un máximo validado en el código. La interfaz sugiere como valor inicial el contenido de la clave dias_prestamo_default de configuracion_sistema (sembrada en 15 días, db/seed.sql; UsuarioPrestamosGestionDTO.diasPrestamoSugerido), pero el backend no lo aplica de oficio si el cliente envía otro valor válido.
- **Método de verificación:** Test (PrestamoServiceTest.crear_conDatosValidos_invocaProcedimientoYRetornaDTO, PrestamoMultaProcedureIntegrationTest — 6 tests de integración reales contra PostgreSQL, no mocks) + **Demonstration** (docs/mediciones/backend/2026-07-29-flujo-prestamo-devolucion-multa-e2e.md).

REQ-F-008 — Registrar devolución

- **Prioridad:** Must
- **Estado:** verificado
- **Fuente:** HU-02 (Cajas), CU-02
- **Módulo/endpoint:** PrestamoController/PrestamoService — POST /api/v1/prestamos/{id}/devolucion (SP sp_registrar_devolucion, @PreAuthorize("hasAnyRole('BIBLIOTECARIO','GERENTE','ADMIN')")); existe una segunda ruta real para el mismo caso de uso, DevolucionController — POST /api/v1/devoluciones/prestamos/{id}/devolucion (@PreAuthorize("hasAnyRole('BIBLIOTECARIO','GERENTE','ADMIN')")), invoca el mismo sp_registrar_devolucion y además soporta registrar daño/pérdida, ver REQ-F-038).
- **Descripción:** registrar la devolución de un préstamo activo, incrementando el stock del libro y generando una multa automáticamente si hubo atraso.
- **Corrección de rol — RESUELTA (hallazgo del Dr. Guerrero, verificado al construir A23; corregida el 2026-09-07, ver OBS-27):** la ruta simple de PrestamoController excluía a BIBLIOTECARIO (la ruta de DevolucionController — la que en la práctica usa el flujo de gestión con posible registro de daño, ver REQ-F-038 — ya lo incluía, a diferencia de REQ-F-007/crear préstamo, donde no existía ninguna ruta alterna accesible para BIBLIOTECARIO). Confirmado con Marlon que era un bug; el @PreAuthorize de la ruta simple se corrigió a hasAnyRole('BIBLIOTECARIO','GERENTE','ADMIN'), verificado con PrestamoControllerSecurityTest.registrarDevolucion y registrarDevolucion_conRolLector_seRechaza. Ambas rutas quedan hoy con el mismo conjunto de roles permitidos.

- **Rationale:** liberar stock y detectar atraso sin intervención manual del bibliotecario (HU-02); la atomicidad de hasta 4 tablas en una sola transacción es exactamente el caso que justifica usar un SP en vez de ORM puro (ADR-006).
- **Criterio de aceptación medible:**
 1. Devolución sin atraso → préstamo DEVUELTO, stock +1, sin multa.
 2. Devolución con atraso → préstamo DEVUELTO, multa PENDIENTE generada, usuario pasa a BLOQUEADO_POR_MULTA. El monto se calcula como $\text{días_de_atraso} \times \text{monto_multa_diaria}$ (`sp_registrar_devolucion.sql`), donde $\text{días_de_atraso} = \text{CEIL}(\text{diferencia_horaria_en_segundos} / 86400)$ (cualquier atraso, aunque sea de horas, cuenta como mínimo 1 día completo) y `monto_multa_diaria` es una clave de `configuracion_sistema` sembrada en 0.50 (`db/seed.sql`), sin tope máximo de monto en el procedimiento.
 3. Doble devolución del mismo préstamo → 409.
 4. Préstamo inexistente → 404.
 5. Falta la clave `monto_multa_diaria` en `configuracion_sistema` (solo relevante con atraso) → 422.
 - Códigos de error del procedimiento (`sp_registrar_devolucion.sql`): LB404 (préstamo no existe), LB409 (préstamo ya devuelto), LB422 (falta configurar `monto_multa_diaria`).
- **Método de verificación:** **Test** (`PrestamoServiceTest.registrarDevolucion_sinAtraso_noGeneraMultas`, `.registrarDevolucion_conAtraso_generaMultas`, `PrestamoMultasProcedureIntegrationTest`, 3 tests) + **Demonstration** (`docs/mediciones/backend/2026-07-29-flujo-prestamo-devolucion-multas-e2e.md`).

REQ-F-009 — Ver préstamos propios

- **Prioridad:** Must
- **Estado:** verificado
- **Fuente:** HU-F02 (Panama), CU-F02
- **Módulo/endpoint:** `PrestamoController/PrestamoService` + `PrestamosLectorComponent` — GET `/api/v1/prestamos/usuario/{id}, .../activos`
- **Descripción:** un LECTOR debe poder ver sus propios préstamos (activos e históricos) desde la interfaz, sin poder consultar los de otro usuario.
- **Rationale:** autoservicio de información sin depender del mostrador (HU-F02); el aislamiento por usuario es un caso concreto de control de acceso, no solo una preferencia de UX.
- **Criterio de aceptación medible:**
 1. LECTOR autenticado ve sus propios préstamos con fecha límite.
 2. LECTOR que intenta pedir los préstamos de otro usuario → acceso denegado.
 3. Sin préstamos registrados → mensaje explícito, no tabla vacía sin contexto (criterio de UI, ver HU-F02).
- **Método de verificación:** **Test** (`PrestamoServiceTest.listarPorUsuario_cuandoLectorPideOtroUsuario_lanzaAccesoDenegado`, `prestamos-lector.component.spec.ts`, 2 tests) + **Demonstration** (`docs/mediciones/frontend/2026-07-30-flujo-front`).

REQ-F-010 — Reporte de libros más prestados

- **Prioridad:** Should
- **Estado:** implementado
- **Fuente:** sin HU/CU dedicada — la matriz marca explícitamente `historia_usuario` y `caso_de_uso` como - para este requisito.
- **Módulo/endpoint:** `PrestamoController/PrestamoService` — GET `/api/v1/prestamos/reportes/libros-mas-prestados` (función `fn_reporte_libros_mas_prestados`)
- **Descripción:** exponer un reporte de los libros con más préstamos registrados, con un límite configurable (default 10).
- **Rationale:** **nota de honestidad** — no hay una HU/CU que documente la necesidad de negocio detrás de este reporte; se infiere que sirve para decisiones de adquisición/gestión del catálogo (rol GERENTE), pero esa motivación no está respaldada por un documento de requisitos específico, solo por la existencia de la función SQL y su test. No se fabrica un rationale más elaborado del que el repositorio realmente sostiene.
- **Criterio de aceptación medible:** sin límite explícito en el request, el sistema aplica un default de 10 resultados (único comportamiento con test de regresión).
- **Método de verificación:** **Test** (`PrestamoServiceTest.reporteLibrosMasPrestados_sinLimite_aplicaDefaultDiez`).

REQ-F-011 — Crear reservación

- **Prioridad:** Must
- **Estado:** verificado

- **Fuente:** HU-03 (Cajas) + HU-F03 (Panama), CU-03 (Cajas) + CU-F03 (Panama)
- **Módulo/endpoint:** `ReservacionController/ReservacionService + ReservacionesComponent` — `POST /api/v1/reservaciones`
- **Descripción:** un usuario autenticado debe poder reservar un libro; si es LECTOR, siempre a su propio nombre (se ignora cualquier `usuarioId` distinto enviado en el request); si es BIBLIOTECARIO/GERENTE, puede reservar a nombre de otro usuario.
- **Rationale:** asegurar un ejemplar sin stock disponible en el momento (HU-03/HU-F03); la resolución del usuario destino ignorando el `usuarioId` del body para un LECTOR es un control de acceso deliberado (mismo patrón que REQ-NF-011 para el rol ejecutor en anulación de multas).
- **Criterio de aceptación medible:**
 1. LECTOR reserva → reservación PENDIENTE a su propio nombre, con fecha de reserva = ahora (zona America/Guayaquil) y fecha límite de retiro calculada como la hora `hora_limite_retiro_reserva` (clave de `configuracion_sistema`, default "18:00" si la clave no está configurada) del día indicado en `fechaRetiro` del request, o del día de hoy si no se envía `fechaRetiro` (`ReservacionService.fromDTO`, `backend-springboot/.../ReservacionService.java:99-125`). **Nota de honestidad (verificado en este commit):** `configuracion_sistema` también sembraba una clave `minutos_reserva` (1440, `db/seed.sql`) que un enunciado previo de este SRS asumía como el mecanismo real de plazo de retiro — se confirmó por búsqueda exhaustiva en `backend-springboot/src/main/java` que **ningún código lee esa clave**; es un parámetro sembrado sin efecto real, no el mecanismo que calcula la fecha límite. PENDIENTE_VERIFICAR_MARLON: confirmar si `minutos_reserva` es vestigial de un diseño anterior y debe eliminarse de `configuracion_sistema`, o si estaba pensado para otro flujo que todavía no lo consume.
 2. BIBLIOTECARIO/GERENTE reserva a nombre de otro usuario → reservación a nombre del usuario indicado.
 3. LECTOR que envía un `usuarioId` distinto al propio → el sistema lo ignora, la reservación se crea igual a su propio nombre.
 4. Libro inexistente → 404.
- **Método de verificación:** `Test (ReservacionServiceTest.crear_*, 4 tests; reservaciones.component.spec.ts, 2 tests)` + **Demonstration** (`docs/mediciones/frontend/2026-07-30-flujo-frontend-prestamos-reservaciones-multas-e2`)

REQ-F-012 — Listar reservaciones propias

- **Prioridad:** Must
- **Estado:** verificado
- **Fuente:** HU-F03 (Panama, **inferida** — la matriz señala explícitamente que el mismo componente cubre creación y listado, sin una HU dedicada solo a listar), CU-F03
- **Módulo/endpoint:** `ReservacionController/ReservacionService` — `GET /api/v1/reservaciones/usuario/{id}`
- **Descripción:** un usuario debe poder listar sus propias reservaciones, con el mismo aislamiento por usuario que REQ-F-009.
- **Rationale:** **nota de honestidad** — no existe una HU separada para “listar reservaciones”; se infiere del hecho de que `ReservacionesComponent` (frontend) implementa ambas operaciones (creación y listado) y de que existe un test de servicio dedicado al aislamiento por usuario. Se documenta como inferido, no como si existiera una HU explícita que no existe.
- **Criterio de aceptación medible:** un LECTOR que intenta pedir las reservaciones de otro usuario recibe acceso denegado (único comportamiento con test de regresión para este requisito específico).
- **Método de verificación:** `Test (ReservacionServiceTest.listarPorUsuario_cuandoLectorPideOtroUsuario_lanzaAccesoDenegado, 1 test)` + **Demonstration** (`docs/mediciones/frontend/2026-07-30-flujo-frontend-prestamos-reservaciones-multas-e2`)

REQ-F-013 — Ver multas propias

- **Prioridad:** Must
- **Estado:** verificado
- **Fuente:** HU-F01 (Panama), CU-F01
- **Módulo/endpoint:** `MultaController/MultaService + MultasComponent` — `GET /api/v1/multas/usuario/{id}`
- **Descripción:** un LECTOR debe poder ver el detalle de sus multas (monto, fecha, estado) sin poder pagarlas ni anularlas desde la UI.
- **Rationale:** autoservicio de información sin exponer acciones reservadas a otros roles (HU-F01) — el lector ve un mensaje indicando que debe acercarse a la biblioteca para regularizar, en vez de un botón de pago que de todas formas el backend rechazaría.
- **Criterio de aceptación medible:**

1. LECTOR ve su multa PENDIENTE con monto, fecha y estado.
 2. La UI del lector no muestra botones “Pagar”/“Anular”.
 3. LECTOR que intenta pedir las multas de otro usuario → acceso denegado.
- **Método de verificación:** **Test** (`MultaServiceTest.listarPorUsuario_cuandoLectorPideOtroUsuario_lanzaAccesoDenegado`, `multas.component.spec.ts`, 2 tests) + **Demonstration** (`docs/mediciones/frontend/2026-07-30-flujo-frontend-pres`)

REQ-F-014 — Pagar multa

- **Prioridad:** Must
- **Estado:** verificado
- **Fuente:** HU-04 (Cajas), CU-04
- **Módulo/endpoint:** `MultaController/MultaService` — `POST /api/v1/multas/{id}/pago` (SP `sp_pagar_multa`)
- **Descripción:** un BIBLIOTECARIO/GERENTE debe poder registrar el pago de una multa PENDIENTE; si era la última multa pendiente del usuario, este vuelve a estado ACTIVO.
- **Rationale:** el lector recupera la posibilidad de pedir préstamos solo cuando ya no tiene ninguna multa pendiente (HU-04); el desbloqueo condicional (verificar que no queden otras multas) es exactamente el tipo de lógica multi-fila que justifica un SP (ADR-006).
- **Criterio de aceptación medible:**
 1. Pago de la única multa pendiente → multa PAGADA, usuario ACTIVO.
 2. Pago con otras multas pendientes → multa pagada cambia a PAGADA, usuario permanece BLOQUEADO_POR_MULTA.
- **Método de verificación:** **Test** (`MultaServiceTest.pagar_invocaProcedimientoYRetornaDTO`, `.pagar_conOtrasMultasF`, `PrestamoMultaProcedureIntegrationTest.pagarMulta_unicaPendiente_desbloqueaUsuario`)
 - **Demonstration** (`docs/mediciones/backend/2026-07-29-flujo-prestamo-devolucion-multa-e2e.md`).

REQ-F-015 — Anular multa

- **Prioridad:** Must
- **Estado:** verificado
- **Fuente:** HU-05 (Cajas), CU-05
- **Módulo/endpoint:** `MultaController/MultaService` — `POST /api/v1/multas/{id}/anulacion` (SP `sp_anular_multa`)
- **Descripción:** solo GERENTE/ADMIN pueden anular una multa registrada por error o por excepción justificada, quedando auditado quién tomó la decisión.
- **Rationale:** corregir sin perder trazabilidad de quién autorizó la excepción (HU-05); el rol ejecutor se resuelve **únicamente** desde la sesión autenticada, nunca desde el body del request — defensa en profundidad reforzada también a nivel de SP (LB422 si el rol no es válido), no solo en el controller (HU-AUTH-07, REQ-NF-010/011).
- **Criterio de aceptación medible:**
 1. GERENTE/ADMIN anula multa PENDIENTE → multa ANULADA, fila nueva en `bitacora_auditoria`.
 2. BIBLIOTECARIO intenta anular → 403 antes de llegar al procedimiento.
 3. BIBLIOTECARIO que envía `"rolEjecutor": "GERENTE"` en el body → el campo se ignora completamente, la operación igual se rechaza.
- **Método de verificación:** **Test** (`MultaServiceTest.anular_conRolGerente_resuelveRolDesdeAuthentication`, `.anular_conRolAdmin_resuelveRolAdmin`, `.anular_sinRolGerenteOAdmin_lanzaAccesoDenegado`, `PrestamoMultaProced`, 2 tests) + **Demonstration** (`docs/mediciones/backend/2026-07-29-flujo-prestamo-devolucion-multa-e2e.md`).

REQ-F-016 — Gestión de préstamos y devoluciones desde la interfaz

- **Prioridad:** Must
- **Estado:** verificado
- **Fuente:** HU-F04 (Panama), CU-F04
- **Módulo/endpoint:** `PrestamosGestionComponent` (frontend) + `PrestamosGestionController` (backend, solo endpoints de apoyo: `buscar-usuario`, `sugerencias-usuarios`, `reserva-activa`, `historial`, todos BIBLIOTECARIO/GERENTE/ADMIN); reutiliza los endpoints de creación/devolución de REQ-F-007/REQ-F-008.
- **Descripción:** el bibliotecario debe poder crear un préstamo y registrar su devolución desde la interfaz web, sin depender de anotaciones manuales.
- **Contradicción real — RESUELTA (hallazgo del Dr. Guerrero, verificado al construir A23; corregida el 2026-09-07, ver OBS-27):** esta descripción asume que un BIBLIOTECARIO puede crear préstamos vía esta interfaz; el endpoint que REQ-F-007 documenta para creación (`POST /api/v1/prestamos`) excluía BIBLIOTECARIO a nivel de `@PreAuthorize`, por lo que un BIBLIOTECARIO recibía 403 al intentar crear un préstamo desde esta

pantalla, contradiciendo el propio criterio de aceptación #1 de este requisito. Confirmado con Marlon que el comportamiento correcto es el que ya asumía esta descripción y el frontend (`prestamos-gestion.component.ts`, que ya trataba a BIBLIOTECARIO como actor válido antes de este fix): se corrigió el `@PreAuthorize` de `PrestamoController.crear` (ver REQ-F-007) en vez de la descripción o el frontend. Para devolución ya existía una ruta real accesible a BIBLIOTECARIO (`DevolucionController`, ver REQ-F-008), y la ruta simple de devolución se corrigió igual (ver REQ-F-008). Los 4 criterios de aceptación de este requisito se cumplen hoy para BIBLIOTECARIO sin excepción.

- **Rationale:** capa de UI sobre la lógica ya especificada en REQ-F-007/REQ-F-008 (HU-F04) — no introduce reglas de negocio nuevas, solo la superficie de interacción.
- **Criterio de aceptación medible:**
 1. Bibliotecario crea préstamo con usuario, libro y días → préstamo registrado (mismo campo `diasPrestamo` obligatorio ≥ 1 días y mismo valor sugerido por defecto de 15 días, ver REQ-F-007).
 2. Bibliotecario registra devolución de un préstamo activo → fila se actualiza con fecha real, botón de devolución desaparece de esa fila.
 3. Préstamos ya devueltos no muestran botón de devolución.
 4. Rechazo del backend (ej. sin stock) → mensaje de error sin cerrar el formulario.
- **Método de verificación:** **Test** (`prestamos-gestion.component.spec.ts`, 2 tests, capa UI sin acceso directo a BD) + **Demonstration** (`docs/mediciones/frontend/2026-07-30-flujo-frontend-prestamos-reservaciones-multas-`

REQ-F-017 — Configuración paramétrica del sistema

- **Prioridad:** Should
- **Estado:** implementado
- **Fuente:** la matriz cita HU-CFG-01/CU-CFG-01, que **no existen** como archivo en `docs/requisitos/historias/` ni `docs/requisitos/casos-de-uso/` (verificado por búsqueda exhaustiva en el repositorio) — se declara como gap, no se inventa su contenido.
- **Módulo/endpoint:** `ConfiguracionSistemaController/ConfiguracionSistemaService` — GET `/api/v1/configuracion` PUT `/api/v1/configuracion/{clave}`
- **Descripción:** solo ADMIN puede listar y editar parámetros clave-valor del sistema (ej. el máximo de renovaciones de un préstamo, ver REQ-F-018) sin necesitar un despliegue nuevo.
- **Rationale:** separa valores operativos que cambian con el tiempo (límites, ventanas) del código fuente, evitando un release solo para ajustar un número; restringido a ADMIN por ser un parámetro de plataforma, no de operación diaria (misma separación de responsabilidades que REQ-F-023, ver ADR-014).
- **Criterio de aceptación medible:**
 1. ADMIN autenticado → GET `/api/v1/configuracion` responde 200 con el listado de claves/valores.
 2. ADMIN actualiza una clave existente vía PUT → 200 con el valor nuevo. **Nota de honestidad:** `ConfiguracionSistemaService.actualizar()` no valida el nuevo valor contra ningún rango ni formato esperado por la clave (acepta cualquier cadena, incluida una no numérica para una clave que un consumidor espera como entero/decimal, ver REQ-F-018); un valor inválido para su clave solo falla más tarde, al leerla (`obtenerValorEntero/obtenerValorDecimal`), no al escribirla.
 3. Rol distinto de ADMIN → 403.
- **Método de verificación:** **Test** (`ConfiguracionSistemaServiceTest`, 6 tests; `ConfiguracionSistemaControllerSecurityTest`, 4 tests).

REQ-F-018 — Renovación de préstamo

- **Prioridad:** Should
- **Estado:** implementado
- **Fuente:** la matriz cita HU-PRE-03/CU-07, que **no existen** como archivo en el repositorio (verificado) — gap declarado.
- **Módulo/endpoint:** `PrestamoController/PrestamoService` (`renovar`, internamente `PrestamoVencidoException/LimiteMaterialReservadoException` — ver **Método de verificación** para el detalle de clase; movidos aquí desde el criterio de aceptación por M15) — POST `/api/v1/prestamos/{id}/renovacion`
- **Descripción:** un LECTOR (solo su propio préstamo) o BIBLIOTECARIO/GERENTE/ADMIN (cualquiera) puede renovar un préstamo activo, siempre que no esté vencido, no haya alcanzado el máximo de renovaciones configurado (REQ-F-017) y no exista una reserva vigente de otro usuario sobre el mismo libro.
- **Rationale:** extiende la fecha límite sin exigir devolver y volver a prestar, con 3 controles de negocio reales (verificados en `PrestamoService.renovar`) para no perpetuar un préstamo indefinidamente ni pisar la reserva de otro lector.

- **Criterio de aceptación medible** (en términos de respuesta HTTP observable, no de nombre de excepción Java — corregido por M15, verificado contra `GlobalExceptionHandler.java` en este commit):
 1. Préstamo activo, no vencido, bajo el límite y sin reserva de otro usuario → 200, fecha límite extendida +`dias_prestamo_default` días (misma clave y mismo valor por defecto que REQ-F-007, 15), contador de renovaciones +1.
 2. Préstamo vencido → 409 Conflict, cuerpo `ProblemDetail` con el detalle del motivo.
 3. Préstamo que ya alcanzó el máximo de renovaciones → 409 Conflict, `ProblemDetail`. El máximo es la clave `max_renovaciones_default` de `configuracion_sistema`, sembrada en 2 (db/seed.sql). **Rango admisible:** ninguno validado en el código — `ConfiguracionSistemaService.actualizar()` acepta cualquier cadena para esta clave (incluida negativa, cero o no numérica); un valor no numérico solo falla, en tiempo de uso, con `IllegalStateException` al leer la clave (`obtenerValorEntero`), no al escribirla vía PUT `/api/v1/configuracion/{clave}` (REQ-F-017).
 4. Libro con reserva vigente de otro usuario → 409 Conflict, `ProblemDetail`.
 5. LECTOR que intenta renovar el préstamo de otro usuario → acceso denegado (403).
- **Método de verificación:** Test (`PrestamoServiceTest`, 6 tests nuevos, casos 50-55 según la matriz) + Inspection (`GlobalExceptionHandler.java`: `PrestamoVencidoException`, `LimiteRenovacionesExcedidoException` y `MaterialReservadoException` mapean las 3 a `HttpStatus.CONFLICT` vía `ProblemDetail.forStatusAndDetail`).

REQ-F-019 — Credencial QR: consulta propia y registro de préstamo con QR

- **Prioridad:** Should
- **Estado:** implementado
- **Fuente:** la matriz cita HU-PRE-04 (no existe como archivo, verificado — gap declarado) y CU-01 (sí existe — “Registrar préstamo”, el mismo caso de uso que ya respalda REQ-F-007. Se reutiliza aquí porque el QR es un mecanismo alternativo de identificación para la misma acción de negocio, no una acción de negocio nueva; se documenta la reutilización explícitamente en vez de asumir sin más que sea un error de la matriz).
- **Módulo/endpoint:** `CredencialQrController/CredencialQrService` — GET `/api/v1/credencial-qr/mi-credencial`; `PrestamoService` (`crear`, `resolverUsuarioId`) — POST `/api/v1/prestamos` (con `credencialQrToken`)
- **Descripción:** cada LECTOR puede obtener la imagen PNG de su propio código QR, generado a partir del token único que Postgres crea al insertar el usuario (`uuid_generate_v4()`, ver docs/diccionario-datos.md); ese token permite identificar al lector al registrar un préstamo sin escribir su usuario/id.
- **Rationale:** agiliza el mostrador (escanear en vez de buscar por nombre/correo); el endpoint de consulta no recibe ningún id en la URL a propósito — se resuelve desde el `Authentication`, así que un LECTOR nunca puede pedir el QR de otro usuario cambiando un parámetro (mismo patrón de aislamiento que REQ-F-009/012/013).
- **Criterio de aceptación medible:**
 1. LECTOR autenticado → GET `/api/v1/credencial-qr/mi-credencial` responde 200 con una imagen PNG.
 2. Rol distinto de LECTOR → 403.
 3. POST `/api/v1/prestamos` con un `credencialQrToken` válido resuelve al usuario dueño de ese token.
- **Método de verificación:** Test (`CredencialQrServiceTest`, 5 tests; `PrestamoServiceTest`, 4 tests nuevos, casos 12-15 según la matriz).

REQ-F-020 — Verificación de correo tras el registro

- **Prioridad:** Must (corregida de Should — ver rationale)
- **Estado:** implementado
- **Fuente:** sin HU/CU dedicada — la matriz marca explícitamente `historia_usuario` y `caso_de_uso` como - para este requisito, mismo patrón que REQ-F-010.
- **Módulo/endpoint:** `AuthController/AuthService/VerificacionCorreoService` — POST `/api/auth/verificar-correo`
- **Descripción:** tras el registro, el usuario queda en estado `PENDIENTE_VERIFICACION` (no puede iniciar sesión) hasta enviar el código de 6 dígitos recibido por correo (TTL configurable, default 10 minutos, almacenado en Redis, sin tabla nueva en Postgres).
- **Rationale:** confirma que el correo registrado existe y es controlado por quien se registró, antes de otorgar acceso — mitiga el registro con correos ajenos o inválidos. **Corrección de prioridad (hallazgo del Dr. Guerrero):** este requisito estaba marcado Should pese a que REQ-F-002 (Must, “Inicio de sesión”) depende de él en la práctica — el criterio 4 de REQ-F-002 (`Usuario INACTIVO/PENDIENTE_VERIFICACION` → 403) no tendría sentido si el estado `PENDIENTE_VERIFICACION` no existiera, y ese estado solo existe porque REQ-F-020 lo introduce en el registro (REQ-F-001). Un requisito Must no puede depender funcionalmente de uno Should: se corrige REQ-F-020 a Must para que la prioridad refleje la dependencia real, no solo el orden cronológico en que se agregó. **Nota de**

honestidad: esto cambia el comportamiento descrito en REQ-F-001 respecto a versiones anteriores de este SRS — el estado inicial tras el registro **ya no es ACTIVO**, es `PENDIENTE_VERIFICACION`; esta versión ya corrigió REQ-F-001 en consecuencia (ver su campo “Depende de”, sección 6 y `CHANGELOG-REQ.md`).

- **Criterio de aceptación medible:**
 1. Código correcto dentro del TTL → 200, usuario pasa a `ACTIVO`.
 2. Código incorrecto o expirado → rechazo, usuario permanece `PENDIENTE_VERIFICACION`.
 3. Usuario `PENDIENTE_VERIFICACION` que intenta iniciar sesión → 403 (mismo criterio que REQ-F-002.4).
- **Método de verificación:** `Test (AuthServiceTest.registroExitoso_dejaAlUsuarioPendienteDeVerificacionYEnviaEmail.verificarCorreo_*, 2 tests nuevos; VerificacionCorreoServiceTest, 5 tests)`.

REQ-F-021 — Consultar notificaciones propias

- **Prioridad:** Should
- **Estado:** implementado
- **Fuente:** sin HU/CU dedicada (matriz: -, -).
- **Módulo/endpoint:** `NotificacionController/NotificacionService` — `GET /api/v1/notificaciones/usuario/{id}`
- **Descripción:** cualquier usuario autenticado puede consultar sus propias notificaciones (préstamo por vencer, multa generada, reserva caducada); un `LECTOR` solo ve las suyas, el resto de roles puede consultar cualquiera (mismo patrón que REQ-F-013).
- **Rationale:** centraliza en la UI las alertas que también intentan enviarse por correo (REQ-F-022a/b/c), para que el usuario no dependa solo de su bandeja de entrada — particularmente relevante ahora que el envío por correo de esas alertas está deshabilitado por defecto (OBS-23) y la notificación in-app es, en la práctica, el único canal que sí llega de forma consistente.
- **Criterio de aceptación medible:** `LECTOR` que pide las notificaciones de otro usuario → acceso denegado (mismo patrón que REQ-F-009/012/013/019).
- **Método de verificación:** `Test (NotificacionServiceTest, 6 tests; NotificacionControllerSecurityTest, 4 tests)`.

REQ-F-022a — Alerta de préstamo por vencer

- **Prioridad:** Should
- **Estado:** implementado
- **Fuente:** sin HU/CU dedicada (matriz: -, -).
- **Módulo/endpoint:** `NotificacionVencimientoScheduler/NotificacionService` — job periódico, sin endpoint propio.
- **Descripción:** el sistema genera una notificación de “préstamo por vencer” mediante un job que corre cada 60 segundos, para todo préstamo dentro de una ventana de anticipación configurable (default 15 minutos antes de la fecha límite).
- **Rationale:** reduce préstamos vencidos por descuido, sin depender de que el bibliotecario o el lector revisen manualmente las fechas.
- **Criterio de aceptación medible:** un préstamo dentro de la ventana de anticipación configurada genera una notificación **una sola vez** (no repetida en cada ejecución del job, que corre cada 60s).
- **Nota de honestidad (verificada en código, 2026-09-07):** el envío real por correo de esta alerta está **deshabilitado por defecto** (`NotificacionService, @Value("${notificaciones.email.habilitado:false}")`), causa raíz documentada en OBS-23: saturación del proveedor SMTP tras el volumen sintético de la rúbrica ADB) — la notificación **sí** se persiste en la tabla `notificaciones` (consultable vía REQ-F-021), pero el correo no se envía salvo que se reactive explícitamente esa clave de configuración. El criterio de aceptación de este requisito es sobre la generación de la notificación, no sobre su entrega por correo, que queda declarada pendiente de reactivación SMTP, no como si funcionara.
- **Método de verificación:** `Test (NotificacionVencimientoSchedulerTest, 3 tests; NotificacionServiceTest.generarAlertaByEmailServiceTest)`.

REQ-F-022b — Alerta de multa generada

- **Prioridad:** Should
- **Estado:** implementado
- **Fuente:** sin HU/CU dedicada (matriz: -, -).
- **Módulo/endpoint:** `PrestamoService (registrarDevolucion)/NotificacionService` — disparado por evento, no por job periódico, sin endpoint propio.

- **Descripción:** el sistema genera una notificación de “multa generada” inmediatamente al registrar una devolución con atraso (no es un job periódico — se dispara en la misma transacción de la devolución).
- **Rationale:** informa al lector de inmediato que quedó BLOQUEADO_POR_MULTA, sin depender de que consulte la app por su cuenta.
- **Criterio de aceptación medible:** toda devolución con atraso (`sp_registrar_devolucion` con `o_hubo_multa = true`) genera exactamente una notificación de multa.
- **Nota de honestidad:** mismo estado que REQ-F-022a — la notificación se persiste, pero el envío por correo depende de `notificaciones.email.habilitado` (deshabilitado por defecto desde OBS-23).
- **Método de verificación:** Test (`PrestamoServiceTest.registrarDevolucion_*`; `NotificacionServiceTest.notificarEmailServiceTest`).

REQ-F-022c — Alerta de reserva caducada

- **Prioridad:** Should
- **Estado:** implementado
- **Fuente:** sin HU/CU dedicada (matriz: -, -).
- **Módulo/endpoint:** `ReservacionScheduler/NotificacionService` — job periódico cada 15 minutos, sin endpoint propio.
- **Descripción:** el sistema genera una notificación de “reserva caducada” mediante `ReservacionScheduler.expirarReservacion` que corre cada 15 minutos y notifica antes de expirar en lote las reservaciones vencidas (ver sección 1.3.1).
- **Rationale:** libera al lector de revisar manualmente si su reserva seguía vigente.
- **Criterio de aceptación medible:** toda reservación PENDIENTE/ LISTA_PARA_RETIRO cuya `fechaLimiteRetiro` ya pasó genera exactamente una notificación antes de expirar.
- **Nota de honestidad:** mismo estado que REQ-F-022a/b — envío por correo condicionado a `notificaciones.email.habilitado` (deshabilitado por defecto desde OBS-23).
- **Método de verificación:** Test (`NotificacionServiceTest.notificarReservaCaducada_*`; `EmailServiceTest`).

REQ-F-023 — Administración de usuarios (rol y estado)

- **Prioridad:** Should
- **Estado:** implementado
- **Fuente:** la matriz cita HU-ADM-01/CU-ADM-01, que **no existen** como archivo en el repositorio (verificado) — gap declarado.
- **Módulo/endpoint:** `UsuarioAdminController/UsuarioAdminService` — GET `/api/v1/admin/usuarios`; PATCH `.../{id}/rol`; PATCH `.../{id}/estado`; POST `/api/v1/admin/usuarios`; DELETE `/api/v1/admin/usuarios/{id}`
- **Descripción:** ADMIN y GERENTE pueden listar el padrón de usuarios (paginado, con filtro); ambos pueden crear cuentas y cambiar rol/estado, pero GERENTE con restricciones reales aplicadas en `UsuarioAdminService` (no un 403 en bloque); solo ADMIN puede dar de baja (soft-delete) una cuenta.
- **Rationale:** separación deliberada entre quién opera el día a día con alcance acotado (GERENTE) y quién administra sin restricciones (ADMIN) — ver ADR-014. **Corrección (hallazgo verificado en código al redactar HU-ADM-01/CU-ADM-01, 2026-09-07):** este requisito describía a GERENTE como de solo lectura del padrón, con 403 en bloque al intentar cambiar rol/estado. `UsuarioAdminController.java` muestra que las tres rutas de escritura (POST, PATCH `.../rol`, PATCH `.../estado`) tienen `@PreAuthorize("hasAnyRole('ADMIN', 'GERENTE'))"` — GERENTE sí puede ejecutarlas a nivel de endpoint. La restricción real vive en `UsuarioAdminService`: GERENTE solo puede crear/asignar los roles LECTOR/BIBLIOTECARIO (`ROLES_GERENTE_PERMITIDOS`), solo puede fijar el estado a ACTIVO/INACTIVO (`ESTADOS_GERENTE_PERMITIDOS`), y en ambos casos únicamente sobre usuarios que él mismo creó (`usuario.getCreadoPor()`). Solo DELETE (baja lógica) es exclusivo de ADMIN a nivel de `@PreAuthorize`. ADR-014 puede describir la intención original de diseño (“solo lectura” para GERENTE); el código implementado es más permisivo que esa descripción — no se corrige el ADR aquí, fuera de alcance de esta tarea de documentación de requisitos.
- **Criterio de aceptación medible:**
 1. ADMIN/GERENTE → GET listado responde 200.
 2. ADMIN cambia rol/estado de cualquier usuario a cualquier valor válido del catálogo → 204.
 3. GERENTE cambia rol de un usuario que él mismo creó, a LECTOR o BIBLIOTECARIO → 204. GERENTE cambia estado de un usuario que él mismo creó, a ACTIVO o INACTIVO → 204.
 4. GERENTE que intenta asignar un rol distinto de LECTOR/ BIBLIOTECARIO, o un estado distinto de ACTIVO/INACTIVO, o actuar sobre un usuario que no creó él mismo → rechazo de acceso (no un 403 de `@PreAuthorize`, sino `AccessDeniedException` lanzada desde el service tras pasar la verificación del endpoint).

5. Rol distinto de ADMIN/GERENTE (ej. BIBLIOTECARIO, LECTOR) en cualquiera de las rutas → 403 en el @PreAuthorize del endpoint.
 6. DELETE /api/v1/admin/usuarios/{id} (baja lógica a INACTIVO) con rol GERENTE → 403 (única ruta exclusiva de ADMIN).
- **Método de verificación:** Test (UsuarioAdminServiceTest, 9 tests; UsuarioAdminControllerSecurityTest, 8 tests) + **Inspection** (lectura directa de UsuarioAdminController.java/UsuarioAdminService.java en este commit para la corrección de rationale/criterio de arriba).

REQ-F-024 — Consultar bitácora de auditoría

- **Prioridad:** Should
- **Estado:** implementado
- **Fuente:** la matriz cita HU-AUD-01/CU-AUD-01, que **no existen** como archivo en el repositorio (verificado) — gap declarado.
- **Módulo/endpoint:** AuditoriaController/AuditoriaService — GET /api/v1/auditoria (filtros usuarioId/modulo/de GET /api/v1/auditoria/resumen (agregación por tabla afectada); GET /api/v1/auditoria/export (exportación CSV, mismos filtros) — **los dos últimos no estaban documentados en versiones anteriores de este SRS**, agregados en esta revisión tras verificar AuditoriaController.java completo.
- **Descripción:** GERENTE/ADMIN (restricción a nivel de clase del controller, @PreAuthorize sobre las tres rutas) pueden consultar de forma paginada y filtrable los eventos registrados en bitacora_auditoria (mismo mecanismo que ya alimenta REQ-NF-007 para autenticación, extendido a otros módulos), consultar un resumen agregado por tabla afectada, y exportar el mismo listado filtrado como CSV.
- **Rationale:** da visibilidad operativa a los mismos datos que hasta ahora solo existían como registro pasivo en la tabla, sin interfaz de consulta.
- **Criterio de aceptación medible:** rol distinto de GERENTE/ADMIN → 403 antes de ejecutar la consulta.
- **Método de verificación:** Test (AuditoriaServiceTest, 4 tests; AuditoriaControllerSecurityTest, 5 tests).

REQ-F-025 — Reporte de índice de morosidad

- **Prioridad:** Should
- **Estado:** implementado
- **Fuente:** sin HU/CU dedicada (matriz: -, -), mismo patrón que REQ-F-010.
- **Módulo/endpoint:** PrestamoController/PrestamoService — GET /api/v1/prestamos/reportes/morosidad (función fn_reporte_indice_morosidad)
- **Descripción:** expone un reporte de los usuarios con más multas/ atrasos, con límite configurable (default 10).
- **Rationale:** **nota de honestidad** — igual que REQ-F-010, no hay HU/CU que documente la necesidad de negocio detrás de este reporte; se infiere un uso gerencial, sin fabricar un rationale más elaborado del que el repositorio realmente sostiene.
- **Criterio de aceptación medible:** sin límite explícito en el request, aplica un default de 10 resultados (único comportamiento con test de regresión).
- **Método de verificación:** Test (PrestamoServiceTest.reporteMorosidad_sinLimite_aplicaDefaultDiez, .reporteMorosidad_conFilas_mapeaProjectionADTO).

REQ-F-026 — Reporte de uso por período

- **Prioridad:** Should
- **Estado:** implementado
- **Fuente:** sin HU/CU dedicada (matriz: -, -).
- **Módulo/endpoint:** PrestamoController/PrestamoService — GET /api/v1/prestamos/reportes/uso (función fn_reporte_uso_por_periodo)
- **Descripción:** expone un reporte de préstamos agrupados por período, con granularidad seleccionable.
- **Rationale:** **nota de honestidad** — mismo caso que REQ-F-010/025, sin HU/CU dedicada.
- **Criterio de aceptación medible:** conjunto cerrado de valores admitidos = {dia, semana, mes} (PrestamoService.GRANULARIDADES_VALIDAS); la comparación es **insensible a mayúsculas** (el valor recibido se aplica .toLowerCase() antes de validar, ej. "MES"/"Mes" se aceptan igual que "mes"); un valor null **no se rechaza**, se normaliza al default "dia"; cualquier otro valor no perteneciente al conjunto cerrado → rechazo explícito (IllegalArgumentException, no un 500 genérico); granularidad válida (o normalizada) → invoca el repositorio con el valor ya en minúsculas.

- **Método de verificación:** `Test (PrestamoServiceTest.reporteUsoPorPeriodo_conGranularidadValida_invocaRepository.reporteUsoPorPeriodo_conGranularidadInvalida_lanzaExcepcion)`.

REQ-F-027 — Exportación a PDF del reporte de morosidad

- **Prioridad:** Should
- **Estado:** implementado
- **Fuente:** sin HU/CU dedicada (matriz: -, -).
- **Módulo/endpoint:** `PrestamoController/ReportePdfService` — `GET /api/v1/prestamos/reportes/morosidad/pdf (fn_reporte_indice_morosidad + PDF en memoria, iText)`
- **Descripción:** genera en memoria (nunca en disco del servidor) el mismo reporte de REQ-F-025 como PDF descargable.
- **Rationale:** mismo dato que REQ-F-025 en un formato apto para imprimir/archivar fuera del sistema; el PDF en memoria evita archivos residuales entre ejecuciones concurrentes de distintos usuarios pidiendo el mismo reporte.
- **Criterio de aceptación medible:** reporte con filas → PDF con los datos; reporte sin filas → PDF con mensaje explícito de “sin datos” (no un PDF vacío sin contexto).
- **Método de verificación:** `Test (ReportePdfServiceTest.generarReporteMorosidad_conFilas_generaPdfConDatosEspañol.generarReporteMorosidad_sinFilas_generaPdfConMensajeVacio)`.

REQ-F-028 — Asistente virtual (Chatbot)

- **Prioridad:** Should
- **Estado:** implementado
- **Fuente:** sin HU/CU dedicada (matriz: -, -).
- **Módulo/endpoint:** `ChatbotController/ChatbotService` (ADR-016) — `POST /api/v1/chatbot/mensajes; GET /api/v1/chatbot/sesiones/{id}/historial`
- **Descripción:** un LECTOR (únicamente, restricción deliberada) puede conversar con un asistente respaldado por Gemini 3.5 Flash Lite; cada mensaje se persiste, la respuesta se genera con grounding real (consulta disponibilidad de libros y reservas del propio usuario antes de responder, para no inventar disponibilidad) y hay un límite de mensajes por usuario en una ventana de tiempo.
- **Rationale:** canal de autoservicio para preguntas frecuentes (horarios, disponibilidad, multas) sin ocupar al personal de mostrador; restringido a LECTOR porque es el actor descrito en el roadmap para este módulo, y para no gastar cuota de la API externa en roles que no lo necesitan. El grounding real (no solo el conocimiento general del modelo) es la decisión central para que el asistente no invente disponibilidad de libros que no existe — ver ADR-016 sobre qué datos se envían a Gemini y por qué no constituye una exposición indebida de información.
- **Criterio de aceptación medible:**
 1. Mensaje válido (1-500 caracteres) de un LECTOR → 200 con la respuesta del asistente.
 2. Mensaje vacío o mayor a 500 caracteres → 400.
 3. Rol distinto de LECTOR o no autenticado → 403.
 4. Sesión inexistente o de otro usuario → 404.
 5. Límite de mensajes excedido → 429. Límite configurable (`app.gemini.rate-limit-max-mensajes / app.gemini.rate-limit-window-seconds`, `ChatbotRateLimiter.java`), sembrado por defecto en **10 mensajes por usuario cada 60 segundos** (ventana fija, contador en Redis con TTL fijado en el primer mensaje de la ventana).
- **Método de verificación:** `Test (ChatbotServiceTest, 8 tests; ChatbotControllerSecurityTest, 5 tests; ChatbotRateLimiterTest, 5 tests)`. **Nota de honestidad:** `ChatbotServiceIntegrationTest` (integración real contra la API de Gemini) está marcado `@Disabled` — requiere `GEMINI_API_KEY` real y consume cuota de la API, se ejecuta solo manualmente, no corre en CI.

REQ-F-029 — Carga y consulta de portada de libro

- **Prioridad:** Should
- **Estado:** implementado
- **Fuente:** HU-LIB-02, CU-LIB-02 (mismo par que REQ-F-006 — es una extensión de la gestión del catálogo, no una acción de negocio nueva).
- **Módulo/endpoint:** `LibroController/LibroService` — `POST /api/v1/libros/{id}/portada (multipart), GET /api/v1/libros/{id}/portada`
- **Descripción:** BIBLIOTECARIO/GERENTE/ADMIN pueden subir la imagen de portada de un libro; cualquier usuario autenticado (LECTOR o superior, mismo alcance que REQ-F-005) puede consultarla, y el portal público también

(PublicoLibroController, sin JWT, ver A6).

- **Rationale:** la portada mejora la identificación visual del catálogo; se sirve por endpoint aparte (no embebida en el DTO del libro) para no inflar el payload del listado con binarios — el DTO solo expone metadata (`tienePortada`, `portadaTipo`, `portadaNombre`).
- **Criterio de aceptación medible:**
 1. Archivo válido (formato y tamaño admitidos, ver más abajo) subido por BIBLIOTECARIO/GERENTE/ADMIN → 200/204, portada persistida como binario en la fila del libro.
 2. GET de un libro con portada → 200 con `Content-Type` dinámico según `portada_tipo` y el binario en el cuerpo.
 3. GET de un libro sin portada o inexistente → 404.
 4. Formatos admitidos y tamaño máximo: validados por `LibroService.validarPortada` contra la clave `max_tamano_portada_mb` de `configuracion_sistema` (sembrada en 2 MB, `db/seed.sql`); archivo que excede el tamaño o cuyo tipo no está entre los admitidos → rechazo explícito (400/422), no un 500 ni una portada truncada. PENDIENTE_VERIFICAR_MARLON: confirmar en `LibroService.validarPortada` la lista exacta de tipos MIME admitidos (no se transcribe aquí sin releer ese método línea por línea, para no adivinar el conjunto exacto).
- **Método de verificación:** Test (`LibroServiceTest.actualizarPortada_*`, 3 tests; `.obtenerPortada_*`, 2 tests; `LibroControllerSecurityTest`, 3 tests nuevos; `LibroPortadaIntegrationTest`, 3 tests de integración real contra Postgres).

REQ-F-030 — Dashboard gerencial

- **Prioridad:** Should
- **Estado:** implementado
- **Fuente:** sin HU/CU dedicada (matriz: -, -).
- **Módulo/endpoint:** `DashboardGerenteComponent` (frontend) — reutiliza GET `/api/v1/prestamos/reportes/libros-mas` (mismo endpoint que REQ-F-010) y otros endpoints de reportes ya especificados (REQ-F-010/025/026); sin endpoint backend propio.
- **Descripción:** GERENTE (y ADMIN, mismo rol que consume los endpoints de reportes) dispone de una vista consolidada que agrega varios reportes existentes (libros más prestados, morosidad, uso) en una sola pantalla, en vez de navegar cada reporte por separado.
- **Rationale:** capa de UI sobre reportes ya especificados — no introduce lógica de negocio ni endpoints nuevos, solo consolida la presentación (mismo patrón que REQ-F-016 sobre REQ-F-007/008).
- **Criterio de aceptación medible:** el componente carga y muestra datos reales de al menos el reporte de libros más prestados sin error, para un usuario con rol GERENTE/ADMIN.
- **Método de verificación:** Test (`dashboard-gerente.component.spec.ts`, 2 tests, capa UI sin acceso directo a BD).

REQ-F-031 — Catálogos maestros (editoriales, idiomas, estados de libro, autores, categorías)

- **Prioridad:** Should
- **Estado:** implementado
- **Fuente:** sin HU/CU dedicada (matriz: -, -).
- **Módulo/endpoint:** `EditorialController` — GET/POST `/api/v1/editoriales`, GET `/api/v1/editoriales/buscar`; `IdiomaController` — equivalente en `/api/v1/idiomas`; `EstadoLibroController` — GET `/api/v1/estados-libro` (sin POST, catálogo cerrado de solo lectura); `AutorController` — equivalente en `/api/v1/autores`; `CategoriaController` — equivalente en `/api/v1/categorias`. **Ampliado en esta revisión** respecto a versiones anteriores de este SRS, que solo citaban los primeros 3 controllers — verificado que `AutorController/CategoriaController` siguen exactamente el mismo patrón.
- **Descripción:** listar (y, salvo `estados-libro`, crear) los catálogos maestros que alimentan la ficha de un libro (editorial, idioma, autor, categoría) y el catálogo de estados de libro (de solo lectura, sin POST — es un enum de dominio, no un catálogo editable por el usuario).
- **Rationale:** evita hardcodear listas en el frontend y permite agregar un editorial/idioma/autor/categoría nuevo sin tocar código.
- **Criterio de aceptación medible:**
 1. GET de cada catálogo → 200 con el listado completo (array plano, sin paginar).
 2. GET `/buscar?q=` (todos salvo `estados-libro`) → coincidencias parciales insensibles a mayúsculas (`findTop5ByNombreContainingIgnoreCase` o equivalente).
 3. POST (todos salvo `estados-libro`) → 201 con la entidad creada.

- **Nota de honestidad — RESUELTA** (hallazgo verificado en código al redactar la actualización del 2026-09-07; corregida el mismo día, ver OBS-27): EditorialController, IdiomaController, AutorController y CategoriaController no tenían ningún @PreAuthorize — ni en el GET ni en el POST. Por el comportamiento por defecto de SecurityConfig (.anyRequest().authenticated()) para todo lo que no está en la lista permitAll(), esto significaba que cualquier usuario autenticado, incluido LECTOR, podía crear un editorial/idioma/autor/categoría nuevo vía POST. Confirmado que no fue una decisión deliberada (ningún ADR ni comentario en el código lo respaldaba); se agregó @PreAuthorize("hasAnyRole('GERENTE','ADMIN')") a los 4 métodos crear(), sin tocar los GET (listar/buscar), que siguen intencionalmente abiertos a cualquier autenticado — ver A23 (matriz de permisos) para el detalle completo.
- **Método de verificación: Test** (CatalogosLibroControllerTest, 4 tests de listar/buscar, sin @PreAuthorize de por medio; más AutorControllerSecurityTest/CategoriaControllerSecurityTest/EditorialControllerSecurityTest/IdiomaControllerSecurityTest, 4 tests cada uno, que sí cargan SecurityConfig real y verifican GERENTE/ADMIN → 201, LECTOR/BIBLIOTECARIO → 403) + **Inspection** (presencia de @PreAuthorize("hasAnyRole('GERENTE','ADMIN')") verificada directamente en el código fuente de los 4 controllers en este commit).

REQ-F-032 — Solicitud de restablecimiento de contraseña

- **Prioridad:** Should
- **Estado:** implementado
- **Fuente:** sin HU/CU dedicada — funcionalidad real sin documento de requisitos previo (mismo patrón que REQ-F-010/020).
- **Módulo/endpoint:** AuthController/AuthService — POST /api/auth/solicitar-reset (permitAll en SecurityConfig, sin JWT).
- **Descripción:** cualquier visitante (sin sesión) puede solicitar un código de restablecimiento de contraseña para un correo; el sistema genera un código de 6 dígitos con TTL de 10 minutos en Redis y lo envía por correo.
- **Rationale:** recuperación de cuenta sin depender de que un administrador resetee la contraseña manualmente.
- **Criterio de aceptación medible:**
 1. Correo registrado → 204, código de 6 dígitos generado en Redis (clave reset-codigo:<correo>, TTL 10 minutos) y envío best-effort por correo (si el envío falla, el código igual queda disponible en Redis para reintentar vía A2).
 2. Redis no disponible al generar el código → ServicioTemporalmenteNoDisponibleException (fail-closed: no se genera un código que luego no se pueda validar de forma confiable).
- **Nota de honestidad (decisión de seguridad verificada en código, no asumida):** este endpoint NO responde igual ante un correo registrado y uno no registrado — AuthService.solicitarReset lanza EntityNotFoundException (→ 404) si el correo no existe en usuarios, antes de generar ningún código. Esto es lo opuesto a la práctica recomendada de “respuesta idéntica siempre” para no permitir enumeración de correos registrados por diferencia de código HTTP (404 vs 204). Se documenta como hallazgo de seguridad real, no como si fuera la decisión correcta — ver A23/OWASP A01 para el detalle; corregirlo (hacer que ambos casos respondan 204 idéntico) queda como recomendación de trabajo futuro, fuera del alcance de esta tarea de documentación.
- **Método de verificación: Inspection** (lectura directa de AuthService.solicitarReset en este commit). PENDIENTE_VERIFICAR_MARLON: no se encontró un test dedicado a este endpoint en la búsqueda realizada para esta revisión — confirmar si existe uno con otro nombre antes de asumir que no hay cobertura de regresión.

REQ-F-033 — Restablecimiento efectivo de contraseña

- **Prioridad:** Should
- **Estado:** implementado
- **Fuente:** sin HU/CU dedicada, mismo patrón que REQ-F-032.
- **Módulo/endpoint:** AuthController/AuthService — POST /api/auth/reset (permitAll, sin JWT).
- **Descripción:** con el código de 6 dígitos recibido (REQ-F-032) y una contraseña nueva, el usuario reemplaza su contraseña sin necesitar sesión activa.
- **Rationale:** cierra el flujo de recuperación de cuenta iniciado en REQ-F-032.
- **Criterio de aceptación medible:**
 1. Código correcto dentro del TTL + contraseña de 8-72 caracteres (ResetPasswordRequestDTO, @Size(min=8, max=72)) → 204, hash actualizado (BCryptPasswordEncoder(12)).
 2. Código incorrecto, expirado, o Redis no disponible al validarlo → CodigoVerificacionInvalidoException (mismo patrón fail-closed que REQ-F-032).
- **Nota de honestidad (verificado en código, ambas preguntas explícitas del hallazgo del Dr. Guerrero):**

1. **Unicidad del token:** es de un solo uso en la práctica — la clave Redis se borra (`redisTemplate.delete(key)`) inmediatamente después de un reseteo exitoso, así que un segundo intento con el mismo código ya no encuentra la clave y falla. No hay, sin embargo, un contador de intentos fallidos sobre ese código dentro de su TTL de 10 minutos (mismo gap de fuerza bruta que A3 declara para reenviar-código).
 2. **Invalidación de sesiones activas:** `AuthService.resetPassword` no invalida ningún token existente — no llama a la blacklist de Redis (REQ-NF-001) ni revoca el `refreshToken` en cookie. Cualquier `accessToken/refreshToken` emitido antes del reseteo sigue siendo válido hasta su expiración natural. Esto es un gap de seguridad real (si la contraseña se resetea porque la cuenta fue comprometida, una sesión ya robada sigue viva) — declarado explícitamente aquí, no asumido como si la invalidación ocurriera.
- **Método de verificación:** **Inspection** (lectura directa de `AuthService.resetPassword` en este commit). PENDIENTE_VERIFICAR_MARLON: mismo caso que REQ-F-032, confirmar existencia de test dedicado antes de asumir que no hay cobertura.

REQ-F-034 — Reenvío de código de verificación de correo

- **Prioridad:** Should
- **Estado:** implementado
- **Fuente:** sin HU/CU dedicada, mismo patrón que REQ-F-032/033.
- **Módulo/endpoint:** `AuthController/AuthService` — `POST /api/auth/reenviar-codigo` (`permitAll`, sin JWT).
- **Descripción:** un usuario recién registrado y aún PENDIENTE_VERIFICACION puede pedir que se regenere y reenvíe el código de verificación de correo (REQ-F-020) si el original expiró (TTL 10 minutos).
- **Rationale:** evita que una cuenta quede bloqueada permanentemente en PENDIENTE_VERIFICACION solo porque el usuario tardó más de 10 minutos en revisar su correo.
- **Criterio de aceptación medible:**
 1. Usuario PENDIENTE_VERIFICACION con correo no verificado → 204, nuevo código de 6 dígitos generado y enviado (mismo mecanismo que REQ-F-020).
 2. Usuario ya verificado (`correoVerificado=true`) o cuyo estado ya no es PENDIENTE_VERIFICACION → rechazo explícito (`IllegalArgumentException`), no reenvía nada.
 3. Correo inexistente → `EntityNotFoundException` (404) — mismo patrón de enumeración de correo que REQ-F-032, no oculto aquí tampoco.
- **Hallazgo de seguridad pendiente (verificado por búsqueda exhaustiva en código, no asumido):** este endpoint no tiene ningún límite de reenvíos ni ventana de tiempo entre solicitudes — `AuthService.reenviarCodigo` no usa `LoginRateLimiter` ni ningún otro limitador; un mismo correo puede disparar reenvíos ilimitados, consumiendo cuota del proveedor SMTP y permitiendo un vector de hostigamiento (spam) hacia la bandeja de entrada de un tercero cuyo correo se conoce. Se declara explícitamente como hallazgo de seguridad pendiente, no se inventa un límite que no existe en el código.
- **Método de verificación:** **Inspection** (lectura directa de `AuthService.reenviarCodigo` y búsqueda de referencias a rate limiters en `AuthController/AuthService` en este commit, sin resultados para este endpoint específico).

REQ-F-035 — Favoritos por usuario

- **Prioridad:** Should
- **Estado:** implementado
- **Fuente:** sin HU/CU dedicada — mencionado en el alcance (1.2) y funciones (2.2) del producto sin requisito formal hasta esta revisión (ver M4).
- **Módulo/endpoint:** `FavoritoController/FavoritoService` — `POST /api/v1/favoritos/{libroId}`, `DELETE /api/v1/favoritos/{libroId}`, `GET /api/v1/favoritos` (paginado), `GET /api/v1/favoritos/todo` (lista completa) — todos `@PreAuthorize("hasRole('LECTOR')")`.
- **Descripción:** un LECTOR puede marcar/desmarcar un libro como favorito y listar únicamente sus propios favoritos; no existe un `usuarioId` en la URL a propósito (el usuario se resuelve del `Authentication`, mismo patrón de aislamiento que REQ-F-009/012/013/019).
- **Rationale:** acceso rápido a libros de interés recurrente del propio lector; exponer `/favoritos/usuario/{usuarioId}` habría permitido a un LECTOR intentar leer favoritos ajenos cambiando el id en la URL (mismo tipo de hallazgo IDOR ya corregido en `PrestamoService.validarAccesoUsuario`) — se evitó ese diseño desde el inicio, documentado explícitamente en el propio código (`FavoritoController.java`).
- **Criterio de aceptación medible:**
 1. LECTOR agrega un libro a favoritos → 201.

2. LECTOR quita un libro de favoritos → 204.
 3. LECTOR lista sus favoritos (paginado o completo) → 200, solo los suyos, sin parámetro de usuario en la URL.
 4. Rol distinto de LECTOR → 403.
- **Método de verificación:** **Inspection** (lectura directa de `FavoritoController.java` en este commit). PENDIENTE_VERIFICAR_MARLON: confirmar el nombre exacto de la clase de test de `FavoritoService/FavoritoController` (no se encontró en la búsqueda realizada con el patrón `Favorito*Test` — puede existir con otro nombre o no existir todavía; no se asume ninguna de las dos sin confirmar).

REQ-F-036 — Sugerencias de adquisición

- **Prioridad:** Should
- **Estado:** implementado
- **Fuente:** sin HU/CU dedicada — mismo caso que REQ-F-035 (ver M4).
- **Módulo/endpoint:** `SugerenciaAdquisicionController/SugerenciaAdquisicionService` — POST `/api/v1/sugerencia` (LECTOR); GET `.../mias` (LECTOR); GET `/api/v1/sugerencias-adquisicion` (GERENTE/ADMIN, filtro estado); PATCH `.../{id}/estado` (GERENTE/ADMIN); GET `.../mas-pedidos` (GERENTE/ADMIN); POST `.../confirmar-adquisicion?isbn=` (GERENTE/ADMIN); GET `.../reporte-pdf` (GERENTE/ADMIN).
- **Descripción:** un LECTOR puede sugerir un libro para que la biblioteca lo adquiera; GERENTE/ADMIN revisan, aprueban o rechazan cada sugerencia, consultan un agrupado de las más pedidas por ISBN, y pueden confirmar en lote la adquisición de todas las PENDIENTE de un ISBN.
- **Rationale:** canaliza peticiones de compra de los propios lectores en vez de depender solo del criterio del personal.
- **Estados del flujo de revisión (conjunto cerrado, verificado en `V2__rbac_normalizado.sql`, CHECK (estado IN ('PENDIENTE', 'APROBADA', 'RECHAZADA'))):** PENDIENTE (inicial) → APROBADA o RECHAZADA, ambas exclusivamente vía PATCH `.../{id}/estado` (`CambioEstadoSugerenciaRequestDTO`, `@Pattern(regexp = "APROBADA|RECHAZADA")`) o, en lote, vía POST `.../confirmar-adquisicion` (todas las PENDIENTE de un ISBN pasan a APROBADA). No existe un cuarto estado “adquirida” separado — se confirmó contra el CHECK real de la tabla, no asumido de la existencia del endpoint `confirmar-adquisicion`.
- **Quién puede cambiar el estado:** exclusivamente GERENTE/ADMIN (`@PreAuthorize("hasAnyRole('GERENTE', 'ADMIN'))`) en `SugerenciaAdquisicionController`; un LECTOR nunca puede aprobar ni rechazar, ni siquiera su propia sugerencia.
- **Criterio de aceptación medible:**
 1. LECTOR crea sugerencia → 201, estado inicial PENDIENTE.
 2. GERENTE/ADMIN cambia estado a APROBADA/RECHAZADA → 200.
 3. LECTOR que intenta cambiar el estado (propio o ajeno) → 403.
 4. GERENTE/ADMIN confirma adquisición por ISBN → todas las PENDIENTE de ese ISBN pasan a APROBADA en lote, respuesta con el conteo de sugerencias confirmadas.
- **Método de verificación:** **Inspection** (lectura directa de `SugerenciaAdquisicionController.java/Service.java` y de la migración `V2__rbac_normalizado.sql` en este commit). PENDIENTE_VERIFICAR_MARLON: mismo caso que REQ-F-035, confirmar existencia y nombre real de la suite de tests antes de citarla.

REQ-F-037 — Portal público de consulta del catálogo sin cuenta

- **Prioridad:** Should
- **Estado:** implementado
- **Fuente:** sin HU/CU dedicada — mencionado como “fuera de alcance... Google Books API” en versiones anteriores de la sección 1.2 sin distinguir que el portal público **sí** existe y es funcionalidad real distinta de esa integración retirada.
- **Módulo/endpoint:** `PublicoLibroController` — GET `/api/publico/libros` (listado con filtros `q/categoriaId/autorId/`); GET `/api/publico/libros/sugerencias` (autocompletado), GET `/api/publico/libros/{id}`, GET `/api/publico/libros` `PublicoCategoriaController` — GET `/api/publico/categorias`. Todo bajo `/api/publico/**`, `permitAll()` en `SecurityConfig.java` — **sin @PreAuthorize en ningún método, por diseño** (el filtro de seguridad ya deja pasar la request antes de llegar al controller).
- **Descripción:** cualquier visitante, sin cuenta ni JWT, puede buscar y ver el catálogo de libros y sus categorías en tiempo real. Reservar, marcar favoritos y crear cuenta siguen requiriendo login.
- **Rationale:** regla de negocio explícita del roadmap (“Rama C”): el catálogo debe poder consultarse públicamente para atraer usuarios antes de registrarse, sin las fricciones de una cuenta.
- **Qué datos expone (verificado leyendo los DTOs reales, no asumido):** `LibroResponseDTO` — la misma estructura que el catálogo autenticado (`LibroController`), incluye `stockTotal/stockDisponible` (inventario agregado de la biblioteca, no ligado a ningún usuario), `precioBase`, metadata de portada, categorías/autores

como texto. NO incluye ningún campo de `Usuario` (no hay join ni referencia a `usuario_id` en este DTO). `CategoriaResponseDTO` expone únicamente `id/nombre`.

- **Criterio de aceptación medible:**
 1. Sin header `Authorization` → 200 en los 5 endpoints públicos.
 2. **Confirmado explícitamente:** la respuesta de ningún endpoint bajo `/api/publico/**` expone stock por usuario individual, ni ningún campo de `nombre/correo/id` de `Usuario`, ni ningún dato personal — el stock que se expone es el inventario agregado de la biblioteca (mismo dato ya público en el mostrador físico), no ligado a ninguna persona.
 3. No existe ningún `POST/PUT/DELETE` bajo `/api/publico/**` — la superficie pública es de solo lectura, por diseño.
- **Método de verificación:** **Test** (`PublicoLibroControllerTest`, suite existente según el javadoc del propio controller) + **Inspection** (lectura directa de `PublicoLibroController.java`, `PublicoCategoriaController.java`, `LibroResponseDTO.java` y `CategoriaResponseDTO.java` en este commit).

REQ-F-038 — Registro de daños de ejemplares

- **Prioridad:** Should
- **Estado:** implementado
- **Fuente:** sin HU/CU dedicada.
- **Módulo/endpoint:** `TipoDanoController` — `GET /api/v1/tipos-dano` (`BIBLIOTECARIO/GERENTE/ADMIN`), `POST/PUT/DELETE` (`ADMIN`, class-level `@PreAuthorize("hasRole('ADMIN')")`); `CategoriaDanoController` — `GET` (`BIBLIOTECARIO/GERENTE/ADMIN`), `POST/PUT/DELETE` (`ADMIN`); `EvidenciaDanoController` — `POST/GET /api/v1/devoluciones/evidencia/{registroDanoId}` (subida/consulta de evidencia fotográfica, multipart, `BIBLIOTECARIO/GERENTE/ADMIN`); `DevolucionController` — `POST /api/v1/devoluciones/prestamo/{id}` (registra la devolución con posible daño, `BIBLIOTECARIO/GERENTE/ADMIN`).
- **Descripción:** al registrar una devolución, el bibliotecario puede declarar que el ejemplar volvió con daño (`CON_DANO`) o que se perdió (`PERDIDO`), adjuntar evidencia fotográfica, y el sistema calcula una multa adicional por el daño (además de la multa por atraso de REQ-F-008, si aplica) sobre `precio_base` del libro.
- **Rationale:** cubre el ciclo completo de vida del ejemplar más allá de préstamo/devolución simple, con costo real para quien lo dañó/perdió.
- **Catálogos y estados relacionados:** `EN_REPARACION/PERDIDO` de `estados_libro` (ver sección 1.3.1) existen en el catálogo, pero **ningún flujo automático los asigna** — `DevolucionService.registrarDevolucion` registra el `RegistroDano` y calcula la multa adicional, sin modificar `estado_libro_id`; ese cambio de estado requiere edición manual posterior del libro (`PUT /api/v1/libros/{id}`). Ya documentado con detalle en la nota de honestidad #2 de la sección 1.3.1 — este requisito la referencia en vez de repetirla.
- **Criterio de aceptación medible:**
 1. Devolución con `estadoDevolucion=CON_DANO` y al menos un daño declarado → se crea `RegistroDano`, multa adicional calculada sobre `precio_base` del libro (best-effort: si el libro no tiene `precio_base`, la multa por daño se calcula en cero, no falla la devolución).
 2. Devolución con `estadoDevolucion=PERDIDO` → mismo registro, sin necesitar `danos` explícitos.
 3. Evidencia fotográfica: `POST` multipart asociado a un `registroDanoId` existente → 201; `GET` del archivo → binario con `Content-Type` real.
 4. Rol distinto de `BIBLIOTECARIO/GERENTE/ADMIN` en cualquiera de estos endpoints → 403; gestión de catálogos (`TipoDano/CategoriaDano`) restringida a `ADMIN` para escritura.
- **Método de verificación:** **Inspection** (lectura directa de `DevolucionService.java`, `TipoDanoController.java`, `CategoriaDanoController.java`, `EvidenciaDanoController.java` en este commit). `PENDIENTE_VERIFICAR_MARLO` confirmar la suite de tests real de este módulo (no confirmada exhaustivamente en esta revisión).

REQ-F-039 — Gestión de proveedores

- **Prioridad:** Should
- **Estado:** implementado
- **Fuente:** sin HU/CU dedicada.
- **Módulo/endpoint:** `ProveedorController` — `GET /api/v1/proveedores` (paginado), `GET .../todo`, `GET .../buscar`, `POST, PUT /{id}` — todos `@PreAuthorize("hasAnyRole('GERENTE','ADMIN')")`.
- **Descripción:** `GERENTE/ADMIN` gestionan el catálogo de proveedores (razón social, contacto), asociable a libros (`libros.proveedor_id`, ver `V36__proveedor_id_en_sugerencia_adquisicion.sql/db/seed.sql` columna `proveedor_id` opcional en `LibroResponseDTO`) y a sugerencias de adquisición.
- **Rationale:** trazabilidad de qué proveedor surtió cada ejemplar, relevante para reposición de stock.

- **Criterio de aceptación medible:**
 1. GERENTE/ADMIN → CRUD completo (GET/POST/PUT) responde 200/201.
 2. Rol distinto → 403 en los 5 endpoints (ningún endpoint de este controller es accesible para LECTOR/BIBLIOTECARIO, a diferencia de otros catálogos maestros como REQ-F-031).
- **Método de verificación:** **Inspection** (lectura directa de `ProveedorController.java` en este commit). PENDIENTE_VERIFICAR_MARLON: confirmar suite de tests real.

REQ-F-040 — Suscripción a disponibilidad de un libro

- **Prioridad:** Should
- **Estado:** implementado
- **Fuente:** sin HU/CU dedicada.
- **Módulo/endpoint:** `SuscripcionDisponibilidadController` — POST `/api/v1/libros/{libroId}/suscripciones`, DELETE `.../suscripciones`, GET `/api/v1/libros/suscripciones/mias` — los 3 con `@PreAuthorize("isAuthenticated()")` (cualquier rol autenticado, no restringido a LECTOR).
- **Descripción:** un usuario autenticado puede suscribirse a un libro sin stock disponible, para recibir notificación cuando vuelva a haberlo (ej. tras una devolución).
- **Rationale:** evita que el usuario tenga que revisar manualmente el catálogo en espera de que un ejemplar quede libre.
- **Criterio de aceptación medible:**
 1. Usuario autenticado se suscribe a un libro → 201.
 2. Usuario cancela su suscripción → 204.
 3. Usuario lista sus propias suscripciones → 200, solo las suyas (resueltas del `Authentication`, mismo patrón de aislamiento que REQ-F-035).
- **Nota de honestidad:** a diferencia de otros módulos de este SRS (ej. REQ-F-035, restringido a LECTOR), este requisito usa `isAuthenticated()` sin restricción de rol — verificado explícitamente en el código, no asumido por analogía con favoritos.
- **Método de verificación:** **Inspection** (lectura directa de `SuscripcionDisponibilidadController.java` en este commit). PENDIENTE_VERIFICAR_MARLON: confirmar si existe el disparo real de la notificación al recuperarse el stock (el controller solo expone alta/baja/listado; el disparo, si existe, viviría en `LibroService` o un scheduler no confirmado en esta revisión) y la suite de tests real.

REQ-F-041 — Pago parcial de multa

- **Prioridad:** Should
- **Estado:** implementado
- **Fuente:** sin HU/CU dedicada.
- **Módulo/endpoint:** `MultaController/MultaService` (`sp_pago_parcial_multa`, columna `multas.monto_pagado`, `fn_pagos_recientes`) — **complementa REQ-F-014** (pago total), que documentaba solo el caso de pago completo.
- **Descripción:** además del pago total de una multa (REQ-F-014), el sistema admite pagos parciales que se acumulan en la columna `monto_pagado` (`ALTER TABLE multas ADD COLUMN monto_pagado NUMERIC(8,2) NOT NULL DEFAULT 0, V16__multas_pago_parcial.sql`); cuando `monto_pagado >= monto`, la multa pasa a PAGADA automáticamente (mismo criterio de desbloqueo del usuario que REQ-F-014).
- **Rationale:** permite a un lector con multas altas regularizar su situación de forma incremental, en vez de exigir el pago total de una sola vez para poder volver a pedir préstamos.
- **Criterio de aceptación medible:**
 1. Pago parcial (`monto < saldo pendiente`) → `monto_pagado` se acumula, multa permanece PENDIENTE, usuario permanece BLOQUEADO_POR_MULTA.
 2. Pago que completa el saldo (`monto_pagado >= monto`) → multa pasa a PAGADA; si era la última multa PENDIENTE del usuario, este vuelve a ACTIVO (mismo criterio que REQ-F-014).
 3. `fn_pagos_recientes` expone el historial de pagos (parciales y totales) recientes para consulta.
- **Método de verificación:** **Inspection** (lectura directa de `V16__multas_pago_parcial.sql` en este commit). PENDIENTE_VERIFICAR_MARLON: confirmar el endpoint HTTP real que expone el pago parcial (si reutiliza POST `/api/v1/multas/{id}/pago` de REQ-F-014 con un campo de monto parcial, o si existe una ruta propia — no se releyó `MultaController.java` completo para esta pregunta específica en esta revisión) y la suite de tests real.

REQ-F-042 — Respaldo y restauración de la base de datos

- **Prioridad:** Should
- **Estado:** implementado
- **Fuente:** decisión arquitectónica, sin HU/CU dedicada.
- **Módulo/endpoint:** BackupController/RespaldoCompletoController (backend, ambos hasRole('ADMIN') en todos sus endpoints) — GET/POST/DELETE /api/v1/admin/backups*, GET/PUT/POST/DELETE /api/v1/admin/respaldo-completo/ microservicio Node backup-service (volcado completo vía pg_dump, ver docs/despliegue/BACKUP.md §6).
- **Descripción:** ADMIN puede programar, ejecutar y descargar respaldos (parciales por tabla vía BackupController, o volcado completo vía RespaldoCompletoController/backup-service); además, la base de producción (Neon) provee restauración punto-en-el-tiempo (PITR) nativa de infraestructura, documentada y probada en vivo.
- **Rationale:** requisito explícito del Bloque ADB (Administración de Bases de Datos) sobre estrategia de backup/recovery con evidencia real, no solo teórica.
- **Criterio de aceptación medible:**
 1. ADMIN programa un backup (POST .../programacion) → 201, ejecutable luego bajo demanda (POST .../{id}/programar) o inmediato (POST .../trigger).
 2. ADMIN descarga un backup existente → binario descargable.
 3. Restauración PITR real probada en Neon: branch backup-recovery-demo-adb restaurada al punto en el tiempo 2026-08-23 15:20 America/Guayaquil, verificada por comparación de conteo de filas entre la rama production y la rama restaurada en el mismo instante (evidencia: docs/mediciones/backup-recovery/, docs/despliegue/BACKUP.md §5.1).
- **Nota de honestidad (verificada en docs/despliegue/BACKUP.md en este commit):** RespaldoCompletoController/backup-service implementa el volcado (pg_dump) pero la restauración desde ese volcado NO existe todavía — el propio documento lo titula “Restauración: NO existe todavía (brecha conocida)” (§6.3). La restauración real verificada de este sistema es la PITR de Neon (infraestructura, no aplicación), no el volcado de backup-service. Ver también A20 para frecuencia/retención.
- **Método de verificación:** **Demonstration** (docs/despliegue/BACKUP.md, docs/mediciones/backup-recovery/*.png) + **Inspection** (BackupController.java, RespaldoCompletoController.java en este commit). PENDIENTE_VERIFICAR confirmar suite de tests unitaria real de BackupService/BackupProgramacionService (existen clases correspondientes en el reporte JaCoCo, docs/mediciones/jacoco/, pero no se confirmó el nombre exacto de sus tests en esta revisión).

3.2 Requisitos no funcionales

Clasificados según las categorías de ISO/IEC/IEEE 29148 aplicables a este proyecto: rendimiento, seguridad, y calidad de software/arquitectura. La gran mayoría de los NF de este sistema son de seguridad porque el foco real de esta entrega (Bloque C.2 de la guía) fue una auditoría OWASP Top 10 en vivo, no una elección arbitraria de énfasis de este documento.

3.2.1 Rendimiento

REQ-NF-003 — TTL configurable del cache del catálogo

- **Prioridad:** Should
- **Estado:** implementado
- **Fuente:** sin HU dedicada (requisito de configuración), CU-LIB-01, ADR-008
- **Módulo:** LibroService — GET /api/v1/libros
- **Descripción:** el cache Redis del listado de libros debe expirar tras un TTL declarado en configuración externa (CACHE_LIBROS_TTL_SECONDS, default 300s), no hardcodeado en Java.
- **Rationale:** antes de esta decisión no existía TTL alguno (cache infinito, solo invalidado manualmente vía @CacheEvict) — riesgo real si los datos subyacentes cambiaran por una vía que el backend no controla (ADR-008).
- **Criterio de aceptación medible:** la clave libros::SimpleKey [] en Redis expira según el TTL configurado, confirmado con redis-cli TTL contra el contenedor real (no simulado).
- **Método de verificación:** **Demonstration** (docs/mediciones/sec/2026-07-21-cache-libros-ttl.md, TTL confirmado en vivo; latencia medida ~170ms en cache miss vs ~30ms en cache hit según docs/arquitectura/ISO25010.md). **Nota:** la prueba de carga formal del Bloque C.1 (k6, 50 VUs) que mide este mismo endpoint bajo concurrencia se ejecutó en un prompt posterior a la redacción original de este requisito — ver docs/mediciones/perf/REPORT.md (p95 caliente 29.79ms, p95 frío 7.96ms, ambos dentro de umbral).

3.2.2 Seguridad

REQ-NF-001 — Revocación inmediata de tokens (blacklist)

- **Prioridad:** Must
- **Estado:** verificado
- **Fuente:** HU-AUTH-03, CU-AUTH-03, ADR-003
- **Descripción:** todo `accessToken` invalidado por `logout` debe quedar en una blacklist de Redis hasta su expiración natural.
- **Rationale:** JWT stateless no tiene revocación nativa — sin esto, un token robado o una sesión cerrada seguiría siendo válido hasta expirar por sí solo (ADR-003, OWASP A07).
- **Criterio de aceptación medible:** clave `blacklist:<jti>` existe en Redis con TTL igual al tiempo restante de expiración del token; una request posterior con ese token es rechazada.
- **Método de verificación:** **Test** (`AuthServiceTest.logoutGuardaTokenEnBlacklist`) + **Demonstration** (`docs/mediciones/sec/owasp/2026-07-30-owasp-a09-fix-logging-autenticacion.md`).

REQ-NF-002 — Cookie `HttpOnly/Secure/SameSite` para el refresh token

- **Prioridad:** Must
- **Estado:** verificado
- **Fuente:** HU-AUTH-04, ADR-012
- **Descripción:** el `refreshToken` debe transportarse exclusivamente en una cookie `HttpOnly`, `Secure`, `SameSite=Strict`, con `path=/api/auth`, nunca en el cuerpo JSON.
- **Rationale:** un secreto de vida más larga que el `accessToken` legible por JavaScript es un vector directo de exfiltración vía XSS; migrarlo a cookie `HttpOnly` lo hace inaccesible a JS por diseño del navegador (ADR-012, OWASP A02). **Corrección de cifra (hallazgo del Dr. Guerrero):** el rationale citaba “7 días” para el `refreshToken` en versiones anteriores de este SRS; `application.yml` (`jwt.refresh-expiration-ms: 10800000`) fija su vida real en **3 horas** (10 800 000 ms), no 7 días — se corrige aquí con el valor verificado directamente en la configuración. **Nota de honestidad heredada de ADR-012:** el `accessToken` (vida corta, `jwt.expiration-ms: 3600000 = 1 hora`, cifra que sí coincidía con versiones anteriores de este SRS) **no** está migrado a cookie todavía — sigue en el cuerpo JSON/memoria del frontend, decisión explícitamente diferida por el impacto en `jwt.interceptor.ts/auth.service.ts`.
- **Criterio de aceptación medible:** la respuesta de login/refresh incluye el header `Set-Cookie: refreshToken=...; HttpOnly; Secure; SameSite=Strict; Path=/api/auth`; el campo `refreshToken` está ausente del cuerpo JSON (`@JsonIgnore` en `TokenResponseDTO`); vida del `accessToken` = 1 hora (`jwt.expiration-ms: 3600000`); vida del `refreshToken` = 3 horas (`jwt.refresh-expiration-ms: 10800000`) — ambos valores de `application.yml`, verificados en este commit.
- **Método de verificación:** **Demonstration** (`docs/mediciones/sec/2026-07-21-cookie-refresh-token.md`, verificado con `curl --include` contra el stack real).

REQ-NF-006 — Rate limiting de intentos de login

- **Prioridad:** Must
- **Estado:** verificado
- **Fuente:** HU-AUTH-05, CU-AUTH-05, `LoginRateLimiter`
- **Descripción:** una combinación correo+IP debe bloquearse temporalmente (429) tras 5 intentos fallidos consecutivos en 900s.
- **Rationale:** dificultar fuerza bruta sin abrir una vía para que un atacante bloquee a la víctima usando su correo desde otra IP — precisamente por eso la clave es correo+IP, no solo correo (HU-AUTH-05, OWASP A07). Este control nació de un hallazgo real de la auditoría de seguridad de esta entrega (ausencia total de rate limiting), no era un requisito preexistente.
- **Criterio de aceptación medible:**
 1. 6.º intento fallido desde el mismo correo+IP en 15 min → 429, sin validar la contraseña.
 2. El bloqueo es por correo+IP: la víctima real, desde su propia IP, no está bloqueada aunque el atacante haya fallado contra su correo desde otra IP.
 3. Un login exitoso resetea el contador de esa combinación a cero.
- **Método de verificación:** **Test** (`LoginRateLimiterTest`, 6 tests; `AuthServiceTest.login*RateLimit*`, 3 tests) + **Demonstration** (`docs/mediciones/sec/owasp/2026-07-30-owasp-a07-fix-rate-limiting-login.md`).

REQ-NF-007 — Auditoría de eventos de autenticación

- **Prioridad:** Must
- **Estado:** verificado
- **Fuente:** HU-AUTH-06, CU-AUTH-06
- **Descripción:** todo LOGIN_OK, LOGIN_FAIL y LOGOUT debe quedar registrado con IP, fecha/hora y usuario/correo, consultable en logs de aplicación y en `bitacora_auditoria`.
- **Rationale:** permitir investigar un incidente de seguridad después de ocurrido, sin depender de que alguien lo reporte en el momento (HU-AUTH-06, OWASP A09). Igual que REQ-NF-006, corrige un hallazgo real de la auditoría (ausencia total de logging de autenticación antes de esta entrega).
- **Criterio de aceptación medible:**
 1. Login exitoso → evento LOGIN_OK con IP, timestamp, id de usuario.
 2. Login fallido → evento LOGIN_FAIL con correo intentado, IP, timestamp (sin id de usuario, porque nunca se resolvió).
 3. Logout → evento LOGOUT con correo, IP, timestamp.
- **Método de verificación:** Test (`AuthServiceTest.loginExitosoReseteaContadorDeRateLimit`, `.loginFallidoIncrementaContadorDeRateLimit`, `.logoutGuardaTokenEnBlacklist` — verifican `bitacoraAuditoriaRepository.save()`) + **Demonstration** (`docs/mediciones/sec/owasp/2026-07-30-owasp-a09-fix-logging-autenticacion.md`).

REQ-NF-010 — RBAC aplicado consistentemente con defensa en profundidad

- **Prioridad:** Must
- **Estado:** verificado
- **Fuente:** HU-AUTH-07, CU-AUTH-07, ADR-010
- **Descripción:** cada endpoint debe verificar el rol del usuario únicamente desde su sesión autenticada, aplicado tanto vía `@PreAuthorize` (Spring Security) como, para las operaciones críticas vía SP, una segunda verificación en el propio procedimiento SQL.
- **Rationale:** ningún usuario debe poder ejecutar una acción reservada a otro rol, ni manipulando el request (HU-AUTH-07); la verificación duplicada (aplicación + base de datos) es defensa en profundidad deliberada, no redundancia accidental (ADR-010, OWASP A01).
- **Criterio de aceptación medible (en términos de respuesta HTTP observable — corregido por M15; el SQLSTATE se movió al campo `modulo_codigo` de la matriz, no queda en el criterio):**
 1. Rol no autorizado en un endpoint restringido → 403 antes de ejecutar lógica de negocio.
 2. Si la verificación de la capa de aplicación se saltara, el SP (ej. `sp_anular_multa`) igual rechaza la operación con 422 Unprocessable Entity y cuerpo `ProblemDetail` (`GlobalExceptionHandler` traduce SQLSTATE LB422 a ese código, ver `docs/trazabilidad/matriz.csv` para la referencia completa al código SQL).
- **Método de verificación:** Test (`MultaServiceTest.anular_sinRolGerenteOAdmin_lanzaAccesoDenegado`, `.listarPorUsuario_cuandoLectorPideOtroUsuario_lanzaAccesoDenegado` + patrones equivalentes en `PrestamoServiceTest/ReservacionServiceTest`)
 - **Demonstration** (`docs/mediciones/sec/owasp/2026-07-30-owasp-a01-control-acceso-roto.md`).**Nota de honestidad** (ya documentada en el informe técnico): existe una asimetría real entre controllers — `LibroController` incluye ADMIN en sus 5 endpoints, `PrestamoController/ReservacionController` no, verificado en vivo al construir `docs/postman/coleccion.json`.

REQ-NF-011 — El rol ejecutor nunca se resuelve desde el body del request

- **Prioridad:** Must
- **Estado:** verificado
- **Fuente:** HU-AUTH-07, `AuthorizationDeniedException` handler
- **Descripción:** el rol usado para autorizar una acción debe resolverse siempre desde el JWT de la sesión (`Authentication`), nunca desde un campo del cuerpo del request (ej. un hipotético `"rolEjecutor"`).
- **Rationale:** cerrar la vía trivial de escalamiento de privilegios que existiría si el backend confiara en cualquier dato de rol enviado por el cliente (HU-AUTH-07, OWASP A01).
- **Criterio de aceptación medible:** un usuario que envía un campo de rol falsificado en el body sigue siendo autorizado/rechazado según su rol real de sesión, no según el valor enviado.
- **Método de verificación:** Test (`MultaServiceTest.anular_sinRolGerenteOAdmin_lanzaAccesoDenegado`) + **Demonstration** (`docs/mediciones/sec/owasp/2026-07-30-owasp-a01-control-acceso-roto.md`).

REQ-NF-012 — TLS en tránsito

- **Prioridad:** Should
- **Fuente:** sin HU dedicada — decisión de entorno, OWASP A02, ADR-015
- **Descripción:** las comunicaciones cliente-servidor deben viajar cifradas (HTTPS), terminando en el proxy (no en el backend Spring Boot), con el backend preparado para reconocer una request como segura cuando venga de ese proxy.
- **Rationale:** proteger credenciales y tokens en tránsito frente a observación de red (OWASP A02); terminar TLS en el proxy en vez del backend evita acoplar la gestión de certificados a la aplicación (ADR-015).
- **Estado real — implementado en el despliegue real de producción, actualizado respecto a versiones anteriores de este SRS** (hallazgo del Dr. Guerrero: versiones previas declaraban esto pendiente sin distinguir el despliegue Docker local del despliegue real en Render): (1) **la decisión de arquitectura** (dónde termina TLS) quedó documentada en ADR-015; (2) **la preparación del backend** (`server.forward-headers-strategy: framework, application.yml:61`) para confiar en X-Forwarded-Proto de un proxy real; (3) **el despliegue real** (`render.yaml`, verificado en este commit) publica **sgb-backend** (Web Service Docker) y **biblora-sgb** (Static Site) sin ningún bloque `domains`: de dominio propio — ambos corren bajo subdominios `*.onrender.com`, donde Render **termina TLS automáticamente en su borde/CDN** con certificados que administra la plataforma (no hay `server.ssl.*` ni certificado propio configurado en este repositorio porque no hace falta: el origen — el contenedor backend — recibe tráfico HTTP plano del proxy de Render, y es exactamente ese proxy el que agrega X-Forwarded-Proto: `https`, la cabecera que el punto (2) ya prepara al backend para confiar). **Lo que sigue sin TLS propio, sin ambigüedad:** el stack de **Docker Compose local** (`docker-compose.yml`, `frontend-angular/nginx.conf`) no activa `server.ssl.*` ni certificado alguno — ese entorno es solo para desarrollo/evaluación local, nunca fue el objetivo de este requisito.
- **Criterio de aceptación medible:** `https://sgb-backend-b058.onrender.com/actuator/health` y `https://biblora-sgb.onrender.com` deben responder con certificado válido (sin advertencias del navegador/curl), emitido y renovado por Render, no por este repositorio; el backend debe reconocer esas peticiones como seguras vía X-Forwarded-Proto (confirmado por `server.forward-headers-strategy: framework`). **Sigue sin cumplirse, sin ambigüedad:** no hay redirección automática HTTP→HTTPS configurada por este repositorio (depende por completo de que Render la fuerce en su borde, no verificado en este commit — PENDIENTE_VERIFICAR_MARLON), ni cabecera **Strict-Transport-Security** propia emitida por el backend.
- **Método de verificación:** **Analysis** (decisión de arquitectura y preparación del backend, revisadas por inspección) — `docs/mediciones/sec/owasp/2026-07-30-owasp-a02-fallo-criptografico.md` (hallazgo original) y `docs/mediciones/sec/owasp/2026-08-10-owasp-a02-fix-tls-transporte.md` (qué se cerró y qué sigue pendiente, con la misma honestidad declarada en el hallazgo original). **TLS real activo end-to-end sigue sin Test ni Demonstration** — no hay stack con certificado real contra el cual verificar.

REQ-NF-013 — Prevención de inyección SQL

- **Prioridad:** Must
- **Estado:** verificado
- **Fuente:** sin HU dedicada, OWASP A03
- **Descripción:** toda consulta (ORM o SP) debe ser parametrizada, sin concatenación de SQL con datos de entrada del usuario.
- **Rationale:** la inyección SQL es uno de los riesgos más severos y mejor entendidos de OWASP Top 10; el patrón híbrido de este proyecto (JPA parametrizado + SPs con parámetros tipados) lo cubre por construcción en ambos mecanismos (ADR-006, OWASP A03).
- **Criterio de aceptación medible:** ninguna consulta del código fuente concatena directamente un valor de entrada del usuario dentro de una cadena SQL.
- **Método de verificación:** **Analysis** — verificación manual puntual durante la auditoría original (`docs/mediciones/sec/owasp/2026-07-30-owasp-a02-fallo-criptografico.md`) **sin test de regresión permanente en el suite** (la propia matriz lo señala explícitamente con un - en la columna de prueba automatizada) — se documenta la ausencia de un test de regresión en vez de implicar que existe uno.

REQ-NF-014a — Content-Security-Policy (backend y frontend)

- **Prioridad:** Should
- **Fuente:** sin HU dedicada, OWASP A05
- **Descripción:** el backend y el frontend deben enviar la cabecera **Content-Security-Policy** para mitigar XSS y clickjacking.

- **Estado real — implementado en ambos lados:** `SecurityConfig.java` (`contentSecurityPolicy(...)`) la envía en las respuestas del backend, confirmado con `curl -I` contra `/actuator/health` real (`docs/mediciones/sec/owasp/2026-07-30-owasp-a05-mala-config-frontend-angular/nginx.conf:10` la envía también, con el modificador `always` — verificado leyendo el archivo directamente en este commit (**hallazgo del Dr. Guerrero:** el gap de CSP del lado frontend que declaraban versiones anteriores de este SRS ya no existe).
- **Criterio de aceptación medible:** las respuestas del backend incluyen `Content-Security-Policy` (cumplido, verificado en vivo); las respuestas del frontend vía Nginx incluyen `Content-Security-Policy` (cumplido, verificado por inspección de `nginx.conf:10` en este commit, sin `Demonstration` nueva contra el contenedor real).
- **Método de verificación:** **Demonstration** (backend: `docs/mediciones/sec/owasp/2026-07-30-owasp-a05-mala-config-frontend-angular/nginx.conf:10` — hallazgo original; `docs/mediciones/sec/owasp/2026-08-10-owasp-a05-fix-csp-stacktrace-swagger-nonroot.md` — cierre por inspección; `docs/mediciones/sec/owasp/2026-08-11-owasp-a05-verificacion-real.md` — verificación real contra Docker) + **Inspection** (frontend: lectura directa de `frontend-angular/nginx.conf:10` en este commit).

REQ-NF-014b — Supresión de stacktraces y mensajes internos en producción

- **Prioridad:** Should
- **Fuente:** sin HU dedicada, OWASP A05
- **Descripción:** el backend en producción no debe exponer stacktraces ni mensajes internos del motor de base de datos en las respuestas de error.
- **Estado real — implementado:** el perfil `prod` de `application.yml` suprime stacktraces/mensajes internos en errores; `GlobalExceptionHandler` traduce toda excepción a `ProblemDetail` (RFC 7807) sin fuga de detalles internos.
- **Criterio de aceptación medible:** ninguna respuesta de error con perfil `prod` activo incluye un stacktrace de Java ni un mensaje interno de PostgreSQL/Hibernate en el cuerpo de la respuesta.
- **Método de verificación:** **Demonstration** (`docs/mediciones/sec/owasp/2026-08-10-owasp-a05-fix-csp-stacktrace-swagger-nonroot.md` — cierre por inspección; `docs/mediciones/sec/owasp/2026-08-11-owasp-a05-verificacion-real.md` — verificación real contra Docker).

REQ-NF-014c — Swagger UI/OpenAPI desactivado en producción

- **Prioridad:** Should
- **Fuente:** sin HU dedicada, OWASP A05
- **Descripción:** el backend en producción no debe exponer Swagger UI/OpenAPI.
- **Estado real — implementado, con un fix intermedio real durante su verificación:** el perfil `prod` de `application.yml` deshabilita Swagger UI/OpenAPI (`springdoc.*.enabled: false`). **Nota de honestidad:** la primera verificación real detectó que `/swagger-ui.html` con `prod` activo devolvía 500 en vez del 404 esperado (`GlobalExceptionHandler` capturaba `NoResourceFoundException` en su `catch-all` genérico) — se corrigió con un `@ExceptionHandler` específico y se reverificó 404 real antes de cerrar este punto (el commit puntual de ese fix ya no es citable por hash, invalidado por la reescritura de historia; ver `evidencia_empirica` de este requisito en la matriz para el anclaje al tag `v1.0.0`).
- **Criterio de aceptación medible:** GET `/swagger-ui.html` y GET `/api-docs` con perfil `prod` activo responden 404, no 500 ni 200 con la documentación real.
- **Método de verificación:** **Demonstration** (`docs/mediciones/sec/owasp/2026-08-10-owasp-a05-fix-csp-stacktrace-swagger-nonroot.md` — cierre por inspección; `docs/mediciones/sec/owasp/2026-08-11-owasp-a05-verificacion-real.md`, incluye la verificación del fix de `NoResourceFoundException`).

REQ-NF-014d — Contenedor del backend sin usuario root

- **Prioridad:** Should
- **Fuente:** sin HU dedicada, OWASP A05
- **Descripción:** el contenedor del backend no debe correr como `root`.
- **Estado real — implementado:** `backend-springboot/Dockerfile` crea el grupo/usuario `spring` (`addgroup -S spring && adduser -S spring -G spring`) y fija `USER spring:spring` antes del CMD — confirmado con `docker exec sgb_backend whoami`.
- **Criterio de aceptación medible:** `docker exec sgb_backend whoami` responde `spring`, nunca `root`.
- **Método de verificación:** **Demonstration** (`docs/mediciones/sec/owasp/2026-08-11-owasp-a05-verificacion-real.md` — verificación real contra Docker).

3.2.3 Calidad de software / arquitectura

REQ-NF-004 — Estrategia híbrida de acceso a datos (ORM + SP)

- **Prioridad:** Must
- **Estado:** verificado
- **Fuente:** HU-01 (Cajas, lado SP) + HU-AUTH-06 (Marlon, lado ORM), ADR-006
- **Descripción:** el CRUD elemental de una sola tabla debe implementarse vía Spring Data JPA; cualquier operación con joins, agregaciones o transacción atómica multi-tabla debe implementarse como procedimiento/función SQL.
- **Rationale:** requisito explícito de la guía del PFC (Bloque A.2), no una preferencia de estilo — ver el análisis completo de alternativas descartadas (ORM puro, SP puro) en ADR-006.
- **Criterio de aceptación medible:** 18 objetos SQL (funciones; ningún PROCEDURE nativo, ver nota de diseño de CATALOGO-SP.md), contados directamente sobre db/procs/*.sql + database/migrations/*.sql en este commit (`grep` por `CREATE (OR REPLACE )?FUNCTION`, 2026-09-07 — cifra corregida respecto a la versión anterior de este SRS, que citaba 7) cubren las operaciones multi-tabla; el resto del acceso a datos usa `JpaRepository` estándar. **Nota de honestidad sobre el alcance de CATALOGO-SP.md:** ese catálogo documenta 17 de los 18 (16 rutinas + el trigger `set_actualizado_en`) porque está **deliberadamente acotado al módulo Préstamos** (su propio título); el objeto 18, `fn_auditoria_generica` (trigger de auditoría genérica, ver `database/migrations/V39_2__fn_auditoria_generica.sql`), es del módulo de auditoría y por eso no aparece en ese catálogo — no es una omisión de este SRS ni de CATALOGO-SP.md, es una diferencia de alcance entre documentos.
- **Método de verificación:** **Test** (`PrestamoMultiProcedureIntegrationTest`, 6 tests contra PostgreSQL real; `AuthServiceTest`, 8 tests contra el lado ORM) + **Demonstration** (`docs/mediciones/backend/2026-07-29-flujo-prestar`

REQ-NF-005 — Esquema de base de datos reproducible

- **Prioridad:** Should
- **Estado:** implementado
- **Fuente:** decisión arquitectónica, ADR-013
- **Descripción:** un evaluador debe poder levantar el sistema completo con datos ya poblados usando un solo comando, sin ejecutar migraciones manualmente.
- **Rationale:** requisito explícito del Bloque B de la guía (reproducibilidad automática); Flyway sigue siendo la fuente de verdad incremental, `db/schema.sql`+`db/seed.sql` es el snapshot de conveniencia para inicialización desde cero (ADR-013).
- **Criterio de aceptación medible:** `docker compose down -v && make up` debe reconstruir el stack completo (44 tablas, contadas por `CREATE TABLE` distintos en `database/migrations/*.sql`, 2026-09-07 — cifra corregida respecto a la versión anterior de este SRS, que citaba 26; `db/schema.sql` cita 31, pero ese snapshot está desactualizado respecto al esquema real, ver OBS-25) desde un volumen vacío, sin pasos manuales adicionales.
- **Nota de honestidad (verificada en este commit, 2026-09-07):** este criterio **no se cumple hoy sin intervención adicional**. OBS-25 (`docs/observaciones/OBSERVACIONES.md`) documenta que un `docker compose down -v && docker compose up` real, sobre un volumen genuinamente vacío, rompe Flyway antes de llegar a la migración V39 (`db/init/01-consolidado.sql` es un snapshot generado solo hasta V13, pero `application.yml` fija `flyway.baseline-version: 37`, así que Flyway saltea V14-V37 como si ya estuvieran aplicadas y V39 falla con `relation "proveedores" does not exist`). Es un bug real, preexistente y ya documentado — no se corrige en esta tarea de documentación de requisitos (fuera de su alcance), pero declararlo “verificado en vivo repetidamente” sin esta salvedad sería inexacto.
- **Método de verificación:** **Demonstration** — el mecanismo de reconstrucción se verificó en vivo repetidamente durante entregas anteriores; la nota de honestidad de arriba es una verificación **más reciente** (2026-09-07) que contradice ese resultado bajo la condición específica de volumen realmente vacío, ver OBS-25.

REQ-NF-008 — PostgreSQL como motor único de base de datos

- **Prioridad:** Should
- **Estado:** implementado
- **Fuente:** decisión arquitectónica, ADR-011
- **Descripción:** el sistema debe usar PostgreSQL 16 como único motor de base de datos, con Row Level Security para aislar datos por rol.
- **Rationale:** RLS nativo (sin el cual el aislamiento por lector dependería de disciplina de código en cada endpoint), PL/pgSQL maduro para los 18 objetos SQL (ver REQ-NF-004), integridad referencial estricta sobre un dominio intrínsecamente relacional — ver comparación completa contra MySQL/MongoDB en ADR-011.
- **Criterio de aceptación medible:** las 44 tablas (cifra corregida respecto a la versión anterior de este SRS,

que citaba 26 — ver REQ-NF-005 para la fuente del conteo), 18 procedimientos/funciones y las políticas RLS de `db/roles-privilegios.sql` corren contra un contenedor `postgres:16-alpine` real.

- **Método de verificación:** **Test** (`PrestamoMultaProcedureIntegrationTest` corre contra PostgreSQL real, no un mock) + **Analysis** (revisión de la decisión arquitectónica en sí, ADR-011).

REQ-NF-009 — Despliegue vía Docker Compose

- **Prioridad:** Should
- **Estado:** implementado
- **Fuente:** decisión arquitectónica, ADR-007
- **Descripción:** los 4 servicios del sistema deben orquestarse con Docker Compose, con imágenes base pinadas por digest sha256 y healthchecks que ordenan el arranque.
- **Rationale:** reproducibilidad de un solo comando sin la complejidad operativa de un orquestador pensado para escalado multi-nodo que este proyecto no necesita (ADR-007, comparación completa contra Kubernetes y despliegue manual).
- **Criterio de aceptación medible:** `docker compose ps` reporta los 4 servicios como `healthy/Up` tras `make up`; las imágenes base están documentadas por digest en `docs/DIGESTS-LOG.md`.
- **Método de verificación:** **Demonstration** — verificado en vivo repetidamente (Status de ADR-007).

REQ-NF-015 — Automatización de CI/CD y documentación de API

- **Prioridad:** Should
- **Estado:** implementado
- **Fuente:** decisión arquitectónica, sin HU/CU dedicada (matriz: N/A - decisión arquitectónica)
- **Módulo:** `.github/workflows/ci.yml` + `Makefile` + `config/OpenApiConfig.java`
- **Descripción:** el sistema debe tener un pipeline de CI que corra build y pruebas de backend/frontend en cada push, un `Makefile` que automatice las tareas repetitivas del equipo (`make up`, `make bench`, `make audit`, `make clean`), y documentación de API autogenerada (OpenAPI/Swagger).
- **Rationale:** reduce el trabajo manual repetitivo del equipo y detecta regresiones antes de que lleguen a `main`, sin depender de que cada integrante recuerde ejecutar los mismos comandos a mano.
- **Criterio de aceptación medible:** cada push a una rama con PR abierta dispara `ci.yml`; `make bench/make audit` ejecutan de verdad (no placeholders, ver OBS-06 en `docs/observaciones/OBSERVACIONES.md`) y generan evidencia versionada en `docs/mediciones/`.
- **Método de verificación:** **Demonstration** — verificado en vivo (ejecuciones reales de `make bench/make audit` con evidencia versionada en `docs/mediciones/perf/` y `docs/mediciones/sec/`).

REQ-NF-016 — Comportamiento ante indisponibilidad de Redis, por servicio

- **Prioridad:** Should
- **Estado:** implementado
- **Fuente:** sin HU dedicada — extiende REQ-NF-001, hallazgo del Dr. Guerrero (M17): la política real de `JwtAuthFilter` es fail-closed, no fail-open como asumía la sección 2.5 en versiones anteriores de este SRS.
- **Descripción:** el sistema debe declarar explícitamente, servicio por servicio, qué ocurre si Redis no responde — no todos los servicios respaldados por Redis se comportan igual, y este requisito documenta cada uno con evidencia de código, no una política uniforme asumida.
- **Rationale:** un supuesto de “todo falla igual” ante Redis caído sería falso y podría llevar a decisiones operativas incorrectas (ej. asumir que el sistema queda abierto cuando en realidad la autenticación se cierra, o viceversa).
- **Criterio de aceptación medible (verificado en código, 4 de 4 servicios con política confirmada — no se asumió ninguno sin leer su manejo de `DataAccessException`):**
 1. `JwtAuthFilter` (verificación de revocación de tokens, REQ-NF-001): **fail-closed** — `catch (DataAccessException e)` responde 401 explícito, ninguna request pasa sin poder confirmar la revocación.
 2. `VerificacionCorreoService.validar` (REQ-F-020): **fail-closed** — `catch (DataAccessException e)` lanza `CodigoVerificacionInvalidoException`, la verificación de correo se rechaza si Redis no responde.
 3. `LoginRateLimiter.estaBloqueado` (REQ-NF-006): **fail-open** — `catch (DataAccessException e)` retorna `false` (no bloqueado); el login sigue intentándose contra la base de datos aunque el rate limit no pueda confirmarse.
 4. `ChatbotRateLimiter.estaBloqueado/registrarMensaje` (REQ-F-028): **fail-open** — mismo patrón que `LoginRateLimiter`: si Redis falla, no se bloquea el mensaje ni se cuenta contra el límite.

- **Método de verificación:** **Inspection** (lectura directa de `JwtAuthFilter.java`, `VerificacionCorreoService.java`, `LoginRateLimiter.java`, `ChatbotRateLimiter.java` en este commit — los 4 catches de `DataAccessException` citados existen literalmente en el código, no se infirieron).

REQ-NF-017 — Umbral de rendimiento bajo carga (p95)

- **Prioridad:** Should
- **Estado:** implementado
- **Fuente:** prueba de carga real, Bloque C.1 de la guía.
- **Descripción:** el endpoint con cache Redis `GET /api/v1/libros` debe responder dentro de un umbral de latencia p95 bajo carga concurrente, distinguiendo cache caliente de cache frío.
- **Rationale:** formaliza como requisito verificable el umbral que ya exigía la guía (Bloque C.1) y que `docs/mediciones/perf/REPORT.md` ya medía, sin que existiera una entrada REQ-NF-XXX dedicada hasta esta revisión.
- **Condición de carga (verificada en el reporte real, no asumida):** 50 VUs (usuarios virtuales) concurrentes, perfil k6 con 10s de ramp-up, 30s sostenido, 10s de ramp-down; escenario `cache_caliente` (siempre `page=0&size=10`, misma key de `@Cacheable`) vs `cache_frio` (página distinta en cada request, PRNG determinista, siempre cache miss).
- **Criterio de aceptación medible (cifras reales agregadas de 5 corridas, docs/mediciones/perf/REPORT.md):**
 1. p95 `cache_caliente` < 200ms — real: **19.49ms** (9929 peticiones).
 2. p95 `cache_frio` < 500ms — real: **7.50ms** (10074 peticiones).
 3. Tasa de error HTTP ≥ 500 : 0% — real: **0.00%** (0 de 20003 peticiones).
- **Método de verificación:** **Demonstration** (`docs/mediciones/perf/REPORT.md`, `k6/libros-listado-test.js`, 5 corridas independientes vía `make bench`, comparación estadística Wilcoxon/Cliff's delta incluida en el mismo reporte).

REQ-NF-018 — Usabilidad medible (System Usability Scale)

- **Prioridad:** Could
- **Fuente:** Bloque C.3 de la guía (evidencia empírica de usabilidad).
- **Descripción:** el sistema debe medirse con el instrumento SUS (System Usability Scale) contra una muestra real de usuarios.
- **Rationale:** complementa la evaluación cualitativa de usabilidad (ISO 25010, ver sección 5) con una métrica cuantitativa estandarizada.
- **Estado real — N=0, muestra retractada:** OBS-08 documenta que una corrida previa fue **retirada** por falta de trazabilidad a un export crudo del instrumento, ausencia de registro de sesiones con hora/duración, y patrones incompatibles con respuestas independientes. El documento vuelve explícitamente a **N=0** en todo el proyecto; la toma de datos real se difiere a la fase de despliegue en producción (`docs/capitulos/10-trabajo-futuro.tex` §SUS). Este requisito no usa ninguna media SUS como válida, porque el propio repositorio no sostiene datos reales de participantes.
- **Criterio de aceptación medible (para cuando exista una muestra real):** media SUS ≥ 75 (umbral convencional de “buena” usabilidad en la literatura SUS) sobre una muestra declarada de participantes reales con consentimiento informado (`docs/etica/consentimientos/plantilla.md`), con export crudo del instrumento versionado y trazable.
- **Método de verificación:** **Analysis** — protocolo, plantilla de consentimiento y pipeline de análisis (`scripts/sus-analysis.ipynb`, validado con datos mock) implementados; **sin Demonstration real todavía** (N=0), declarado explícitamente, no simulado como si existiera.
- **Condición de cierre:** se cierra cuando se complete la recopilación de N226515 respuestas SUS v00e1llidas en el despliegue de producción real, con export crudo del instrumento versionado y trazable, y análisis estadístico completado (media, IC95%, desviación estándar) — conforme a lo planificado en `docs/capitulos/10-trabajo-futuro.tex` §SUS y documentado en `docs/observaciones/OBSERVACIONES.md` (OBS-08).

REQ-NF-019 — Accesibilidad del frontend (Lighthouse)

- **Prioridad:** Should
- **Estado:** implementado
- **Fuente:** Bloque C.5 de la guía, auditoría Lighthouse real.
- **Descripción:** el frontend debe cumplir un umbral de accesibilidad medido con Lighthouse, perfil móvil con throttling Slow 4G.

- **Rationale:** verificación automatizada de accesibilidad básica (contraste, roles ARIA, navegación) sin depender de una revisión manual exhaustiva.
- **Nota de honestidad sobre el estándar usado (verificado en el reporte real, no asumido):** docs/mediciones/lighthouse/REPORT.md no cita ningún nivel WCAG específico — el score reportado es la categoría “Accessibility” propia de Lighthouse (basada en reglas `axe-core`), no una certificación de conformidad WCAG 2.1 AA ni de ningún otro nivel. Este requisito no asume WCAG 2.1 AA porque el reporte que lo respalda no lo declara.
- **Criterio de aceptación medible (cifra real, lhci-20260731-0300.json):** categoría Accessibility ≥ 90 — real: **95**, cumple. Auditoría específica que sí resta puntos dentro de esa categoría: `color-contrast` (score 0, “Background and foreground colors do not have a sufficient contrast ratio” en al menos un elemento) — hallazgo real no corregido en esta revisión (fuera de alcance de una tarea de documentación de requisitos).
- **Método de verificación: Demonstration** (docs/mediciones/lighthouse/REPORT.md, lhci-20260731-0300.json, perfil móvil + Slow 4G, Lighthouse v12.x).

REQ-NF-020 — SEO del portal público (Lighthouse)

- **Prioridad:** Should
- **Estado:** implementado
- **Fuente:** Bloque C.5 de la guía, mismo informe que REQ-NF-019.
- **Descripción:** el frontend debe cumplir un umbral de SEO medido con Lighthouse (mismas condiciones que REQ-NF-019).
- **Rationale:** relevante específicamente para A6 (portal público sin cuenta) — sin indexabilidad razonable, el portal público pierde parte de su propósito de atraer usuarios antes del registro.
- **Estado real — NO cumple el umbral, declarado sin ambigüedad:** categoría SEO ≥ 90 exigido — real: **82**, no cumple.
- **Causas identificadas (extraídas del propio JSON del reporte, no interpretadas a mano):**
 1. `meta-description` (score 0): `index.html` del build de Angular no tiene una etiqueta `<meta name="description">` — confirmado manualmente contra el archivo real.
 2. `robots-txt` (score 0): GET `/robots.txt` responde 200 pero con el `index.html` de la SPA (por el fallback de rutas de Angular/nginx), no un `robots.txt` real — Lighthouse lo rechaza como inválido.
- **Criterio de aceptación medible:** categoría SEO ≥ 90 (no cumplido hoy); ambas causas identificadas arriba son corregibles sin cambios de arquitectura (agregar `<meta name="description">` al `index.html`; servir un `robots.txt` real desde `nginx.conf` antes del fallback `try_files`) — quedan como trabajo futuro, no se corrigen en esta tarea de documentación de requisitos.
- **Método de verificación: Demonstration** (docs/mediciones/lighthouse/REPORT.md, lhci-20260731-0300.json).

REQ-NF-021 — Objetivos de respaldo y recuperación (frecuencia, retención, RPO/RTO)

- **Prioridad:** Must
- **Estado:** implementado
- **Fuente:** Bloque ADB (Administración de Bases de Datos) de la guía, docs/despliegue/BACKUP.md.
- **Descripción:** el sistema debe declarar objetivos concretos de respaldo y recuperación para la base de datos de producción (Neon).
- **Rationale:** sin objetivos declarados, “hacer respaldo” no es verificable — este requisito fija los números reales que el equipo ya documentó y verificó en docs/despliegue/BACKUP.md, en vez de dejarlos dispersos solo en ese documento operativo.
- **Criterio de aceptación medible (valores reales verificados en docs/despliegue/BACKUP.md en este commit, ninguno inventado):**
 1. **Ventana PITR (Point-In-Time Recovery) de Neon:** 6 horas, tope de 1 GB de cambios acumulados — límite fijo del plan Free, no configurable (confirmado contra neon.com/docs/introduction/plans, vigencia verificada agosto 2026).
 2. **Retención mínima obligatoria de este proyecto:** 30 días posteriores a la defensa (2026-08-17) → mínimo hasta el **2026-09-16** — vigente al momento de este commit (2026-09-07). Durante esa ventana: no eliminar los servicios de Render, no eliminar el proyecto/base de Neon, no eliminar la base de Upstash.
 3. **Respaldo a nivel de aplicación** (`configuracion_respaldo`, `V34__sistema_dual_respalDOS.sql`): frecuencia por defecto `frecuencia_horas = 6`, retención por defecto `dias_retencion = 14` (ambos configurables por ADMIN vía `BackupController`, sin rango validado al escribir — mismo patrón de `ConfiguracionSistemaService` que REQ-F-017/018).

4. **RPO/RTO como métricas formales con ese nombre:** <PENDIENTE_CONFIRMAR> — no se encontró ningún documento del repositorio que declare un RPO o RTO explícito en minutos/horas; lo más cercano son los puntos 1-3 de arriba (ventana PITR y retención), que acotan el RPO/RTO de forma indirecta sin nombrarlos así. No se inventa una cifra de RPO/RTO que el repositorio no declara.
- **Nota de honestidad adicional:** docs/despliegue/BACKUP.md §5.2 documenta que, al 2026-08-31, la ventana PITR de este proyecto Neon específico estaba **deshabilitada/no disponible** en la práctica (no solo acotada a 6h) al intentar un segundo ejercicio de restauración — declarado explícitamente en ese documento en vez de repetir el ejercicio como si hubiera funcionado igual que la primera vez (2026-08-23, sí exitosa).
- **Método de verificación:** **Demonstration** (docs/despliegue/BACKUP.md completo, docs/mediciones/backup-recovery,

REQ-NF-022 — Protección de datos personales (minimización, consentimiento, ausencia de exposición)

- **Prioridad:** Must
- **Estado:** implementado
- **Fuente:** Bloque F de la guía (ética de datos), docs/etica/ETHICS.md.
- **Descripción:** el sistema debe minimizar los datos personales que recolecta y envía a terceros, y no debe exponer accidentalmente datos sensibles (contraseñas) vía la API.
- **Rationale:** obligación básica de cualquier sistema que almacena datos de personas reales (nombre, apellido, correo de lectores/personal de una biblioteca institucional).
- **Criterio de aceptación medible (verificado en ETHICS.md y en el código citado por ese documento, en este commit):**
 1. **Minimización de campos:** el esquema no recoge datos personales más allá de **nombre/apellido/correo** — sin datos sensibles (salud, biométricos) sin relación con la operación de una biblioteca.
 2. **No exposición de password_hash:** verificado por auditoría completa del backend — @JsonIgnore en la entidad, ningún DTO de respuesta incluye el campo, ningún controller retorna la entidad Usuario directamente.
 3. **Minimización hacia el proveedor externo de IA (Gemini):** verificado leyendo GeminiClient.java y ADR-016 en este commit — **nunca** se envían credenciales, tokens JWT, contraseñas, identificación personal (cédula) ni el correo del usuario; lo que sí se envía es el texto del mensaje del usuario (máx. 500 caracteres), el historial de esa sesión de chat, y un prompt de sistema con entradas curadas de **base_conocimiento** y resultados de disponibilidad de catálogo (ya públicos vía A6/REQ-F-037). **SesionChat/MensajeChat** solo guardan **usuarioId** (clave foránea numérica), no el correo ni el nombre.
- **Nota de honestidad — hueco real, no inventado:** ni ETHICS.md ni ningún ADR del repositorio declaran una **política de retención de bitacora_auditoria** (cuánto tiempo se conservan los eventos) ni un **mecanismo de supresión de datos a solicitud del titular** (derecho de cancelación/oposición bajo la Ley Orgánica de Protección de Datos Personales de Ecuador). Verificado por búsqueda exhaustiva (**retención, supresión, LOPD, titular**) sin resultados en docs/etica/, docs/adr/ ni el código. PENDIENTE_VERIFICAR_MARLON: definir explícitamente (a) el período de retención de **bitacora_auditoria** y (b) el procedimiento operativo (manual o automatizado) para atender una solicitud de supresión de datos personales, antes de poder declarar este requisito cumplido en su totalidad — hoy solo la minimización (puntos 1-3) está verificada, la retención/supresión no.
- **Método de verificación:** **Inspection** (docs/etica/ETHICS.md, ADR-016, GeminiClient.java, Usuario.java, en este commit).

REQ-NF-023 — Política de contraseñas

- **Prioridad:** Should
- **Estado:** implementado
- **Fuente:** sin HU dedicada, verificado en código.
- **Descripción:** el sistema debe declarar su política real de contraseñas, sin agregar reglas de composición que el código no aplica.
- **Rationale:** una política de contraseñas documentada más estricta que la implementada induciría a error a quien audite el sistema asumiendo protecciones que no existen.
- **Criterio de aceptación medible (único control real, verificado en RegistroRequestDTO.java/ResetPasswordRequest en este commit — no se agregan reglas de composición que no existan):**
 1. Longitud mínima de 8 caracteres (@Size(min = 8)), sin máximo en el registro; en el reseteo (REQ-F-033), rango 8-72 caracteres (@Size(min = 8, max = 72), el máximo es el límite de entrada de BCrypt, no una regla de negocio).

2. **No existe** ninguna regla de composición (mayúscula, minúscula, número, símbolo obligatorio) en ningún DTO ni servicio de autenticación — se declara explícitamente su ausencia, no se asume que exista por convención.
 3. Hash con `BCryptPasswordEncoder(12)` (costo 12), nunca texto plano.
- **Sesiones activas al cambiar contraseña:** **no existe** un endpoint de “cambiar mi contraseña” para un usuario ya autenticado — el único camino de cambio de contraseña es el flujo de reseteo sin sesión (REQ-F-032/033). Como ya documenta REQ-F-033, ese reseteo **no invalida sesiones/tokens activos** — mismo gap, no repetido en detalle aquí.
 - **Método de verificación:** **Inspection** (`RegistroRequestDTO.java`, `ResetPasswordRequestDTO.java`, `SecurityConfig.java` en este commit).

REQ-NF-024 — Interfaces externas consumidas (SMTP y API de Gemini)

- **Prioridad:** Should
- **Estado:** implementado
- **Fuente:** decisión arquitectónica, `render.yaml`, ADR-016.
- **Descripción:** a diferencia de la única interfaz **expuesta** por este sistema (la API REST propia, sección 3.3), el backend **consume** dos interfaces externas: SMTP (vía Brevo, para correo transaccional) y la API de Gemini (para el chatbot). Este requisito formaliza ambas — ver también M26 (sección 3.3 corregida para distinguir “expuesta” de “consumida”).
- **Rationale:** una auditoría de superficie de ataque/dependencias externas necesita ambas interfaces documentadas, no solo la propia API REST.
- **Criterio de aceptación medible, por interfaz (verificado en código en este commit):**
 1. **SMTP/Brevo** (`EmailService.java`): protocolo SMTP (host/puerto configurables vía `SMTP_HOST/SMTP_PORT=587` en `render.yaml`, `sync: false` — credenciales no versionadas); autenticación usuario/contraseña (`SMTP_USER/SMTP_PASS`, también `sync: false`); ante indisponibilidad del proveedor, `EmailService` captura `MessagingException/MailException` y retorna `false` (no lanza excepción hacia el llamador) — el flujo que lo invoca (ej. `AuthService.solicitarReset`) continúa de forma best-effort, sin romper la operación principal.
 2. **API de Gemini** (`GeminiClient.java`): protocolo HTTP/JSON directo (sin SDK), autenticación por API key (`GEMINI_API_KEY`, `render.yaml sync: false`, nunca registrada en logs — ver ADR-016); ante indisponibilidad, reintenta hasta 2 veces y distingue el tipo de fallo: 429 Too Many Requests/timeout de red → mensaje de “saturado” tras el reintento; error 4xx/5xx del servidor de Gemini → mensaje de fallback genérico inmediato (sin reintento adicional en ese branch) — nunca deja la conversación sin respuesta ni propaga una excepción cruda al usuario.
- **Método de verificación:** **Inspection** (`EmailService.java`, `GeminiClient.java`, `render.yaml`, ADR-016 en este commit).

3.3 Requisitos de interfaz externa

Corrección de alcance (hallazgo del Dr. Guerrero, M26): versiones anteriores de esta sección afirmaban que el sistema “expone una única interfaz externa real” — esa frase mezclaba dos conceptos distintos: la interfaz que este sistema **expone** hacia sus clientes (el frontend, o cualquier consumidor de la API) y las interfaces que este sistema **consume** de terceros. Ambas existen y son reales; se separan aquí en vez de seguir mezclándolas bajo “única interfaz externa”.

3.3.1 Interfaz expuesta por este sistema El backend expone una única interfaz **hacia sus clientes**: una **API REST sobre HTTP/JSON**, documentada automáticamente vía springdoc-openapi (Swagger UI en `/swagger-ui.html`, ver ADR-001) y consumida por el frontend Angular. No existen requisitos de interfaz externa con ID propio en `docs/trazabilidad/matriz.csv` para esta API — cada endpoint concreto ya está trazado como parte del requisito funcional que lo usa (columna `endpoint_api` de la matriz, sección 3.1 de este documento). **Cifra recontada en esta revisión (hallazgo del Dr. Guerrero, 2026-09-07)** — versiones anteriores de este SRS citaban 19→44 endpoints y 5→15 `@RestController` en pasos sucesivos, ambas ya desactualizadas frente al código real de este commit: hay **31 clases `@Controller`** en `backend-springboot/src/main/java/com/uteq/backend/controller/` (`find ... -name "*Controller.java" | wc -l`, 30 con lógica de negocio real + `TestController`, un endpoint de humo sin lógica de negocio) y **143 combinaciones método+ruta** (`@GetMapping/@PostMapping/@PutMapping/@PatchMapping/@DeleteMapping`: `86+37+7+4+9`, `grep -rhoE` sobre el mismo directorio), contadas directamente sobre el código fuente en este commit,

no inferidas. **Nota de honestidad:** esta cuenta no se propagó a `docs/informe-entrega-3.tex` (sección “Estado del sistema”) ni a `docs/postman/coleccion.json` (que sigue citando 39 requests) — ambos quedan fuera del alcance de esta actualización del SRS, así que pueden estar desactualizados en la misma dirección que este documento lo estaba antes de esta versión; no se corrigen aquí para no tocar archivos fuera del alcance de esta tarea.

Contrato general: request/response en JSON; autenticación vía header `Authorization: Bearer <accessToken>` (excepto `/api/auth/refresh`, que usa la cookie `refreshToken`); errores en formato `ProblemDetail` (RFC 7807) vía `GlobalExceptionHandler`, sin fuga de detalles internos (stacktraces, mensajes de motor de base de datos) al cliente.

3.3.2 Interfaces externas consumidas por este sistema A diferencia de 3.3.1, el backend **consume** dos interfaces de terceros reales, declaradas en `render.yaml` y verificadas en el código: **SMTP (vía Brevo)** para correo transaccional, y la **API de Gemini** para el chatbot. Ambas se formalizan con criterio de aceptación propio en **REQ-NF-024** (sección 3.2.3, Bloque 5/A25 de esta actualización) — esta subsección solo las referencia para que la sección 3.3 quede completa sin duplicar el contenido ya redactado allí.

3.4 Anexos

A23 — Matriz normativa de permisos rol × operación Construida leyendo directamente el `@PreAuthorize` (o su ausencia) de los 31 controladores, 2026-09-07 — no inferida de la documentación de cada requisito. **Sí** = permitido, **-** = rechazado (403), **pub.** = sin autenticación (`permitAll`), **auth.** = cualquier rol autenticado (`isAuthenticated()` o sin anotación, mismo efecto por el default de `SecurityConfig`).

Controller	Operación(es)	LECTOR	BIBLIOTECARIO	GERENTE	ADMIN
<code>AuthController</code>	registro, login, refresh, reset, verificar correo, logout	pub.	pub.	pub.	pub.
<code>PublicoLibroController</code>	listar/detalle/portada (GET)	pub. Sí	pub. Sí	pub. Sí	pub. Sí
<code>LibroController</code>	pendientes/lookup-isbn (GET)	—	Sí	Sí	Sí
<code>LibroController</code>	crear/editar/eliminar/subir portada	—	Sí	Sí	Sí
4 catálogos maestros (REQ-F-031)	listar/buscar (GET)	auth.	auth.	auth.	auth.
4 catálogos maestros (REQ-F-031)	crear (POST) — nota 1 (corregido 2026-09-07)	—	—	Sí	Sí
<code>EstadoLibroController</code>	listar (GET, sin POST)	auth.	auth.	auth.	auth.
<code>FavoritoController</code>	agregar/quitar/listar propios	Sí	—	—	—
<code>SugerenciaAdquisicionController</code>	crear/listar propias	Sí	—	—	—
<code>SugerenciaAdquisicionController</code>	listar todas/cambiar estado/más-pedidos/confirmar/PDF	—	—	Sí	Sí
<code>SuscripcionDisponibilidadController</code>	cancelar/listar propias	auth.	auth.	auth.	auth.
<code>PrestamoController</code>	crear / devolución simple (ver nota 2, corregido 2026-09-07)	—	Sí	Sí	Sí
<code>DevolucionController</code>	registrar devolución completa (con daño), historial	—	Sí	Sí	Sí
<code>PrestamoController</code>	renovación	Sí (solo propio)	Sí	Sí	Sí
<code>PrestamoController</code>	listar por usuario / activos (ver nota 3)	Sí (solo propio)	Sí	Sí	—
<code>PrestamoController</code>	reportes (morosidad/uso/inventario/vencidos/categorías, JSON y PDF)	—	—	Sí	Sí
<code>PrestamosGestionController</code>	buscar-usuario/sugerencias/reserva-activa/historial	—	Sí	Sí	Sí
<code>ReservacionController</code>	crear (ver nota 3)	Sí (propio)	Sí	Sí	—
<code>ReservacionController</code>	hoy/próximas	—	Sí	Sí	Sí
<code>ReservacionController</code>	cambiar estado (aceptar/rechazar)	Sí	Sí	Sí	Sí
<code>ReservacionController</code>	listar por usuario (ver nota 3)	Sí (propio)	Sí	Sí	—
<code>ReservacionesGestionController</code>	buscar-usuario/historial	—	Sí	Sí	Sí
<code>MultaController</code>	ver por usuario/detalle	Sí (propio)	Sí	Sí	Sí

Controller	Operación(es)	LECTOR	BIBLIOTECARIO	GERENTE	ADMIN
MultaController	pago	—	Sí	Sí	Sí
MultaController	anulación, reportes	—	—	Sí	Sí
CredencialQrController	mi-credencial	Sí	—	—	—
ChatbotController	mensajes/historial	Sí	—	—	—
NotificacionController	ver por usuario	Sí (propio)	Sí	Sí	Sí
ConfiguracionSistemaController	listar/actualizar (REQ-F-017)	—	—	—	Sí
UsuarioAdminController	listar padrón	—	—	Sí	Sí
UsuarioAdminController	crear/cambiar rol/cambiar estado (con alcance limitado para GERENTE, ver REQ-F-023)	—	—	Sí (acotado)	Sí
UsuarioAdminController	eliminar (baja lógica)	—	—	—	Sí
AuditoriaController	listar/resumen/export (REQ-F-024)	—	—	Sí	Sí
TipoDanoController/CategoriaController	listar/buscar	—	Sí	Sí	Sí
TipoDanoController/CategoriaController	eliminar	—	—	—	Sí
EvidenciaDanoController	subir/consultar evidencia	—	Sí	Sí	Sí
ProveedorController	todo (CRUD)	—	—	Sí	Sí
BackupController/RespaldoCompletoController	completo	—	—	—	Sí
TestController	/protegido (endpoint de humo, sin lógica de negocio)	auth.	auth.	auth.	auth.

Notas de esta matriz (asimetrías reales encontradas, ninguna oculta):

1. **Hueco real, RESUELTO el 2026-09-07 (ver OBS-27):** POST en `AutorController/CategoriaController/EditorialController` no tenía ningún `@PreAuthorize` — cualquier usuario autenticado, incluido LECTOR, podía crear entradas en estos catálogos maestros. No se encontró ningún ADR ni comentario en el código que declarara esto como intencional (ver también REQ-F-031). Se corrigió agregando `@PreAuthorize("hasAnyRole('GERENTE','ADMIN'))"` a los 4 métodos `crear()` — mismo conjunto de roles que ya usa `PrestamoController` para operaciones administrativas de catálogo — verificado con tests reales (`AutorControllerSecurityTest`, `CategoriaControllerSecurityTest`, `EditorialControllerSecurityTest`, `IdiomaControllerSecurityTest`, 4 tests cada uno: GERENTE/ADMIN 201, LECTOR/BIBLIOTECARIO 403). Los GET (listar/buscar) de estos 4 controllers siguen abiertos a cualquier autenticado — eso sí es intencional, no se tocó.
2. **Contradicción real entre documentación y código, RESUELTA el 2026-09-07 (ver OBS-27):** `PrestamoController.crear/ .registrarDevolucion` (la ruta simple) excluían BIBLIOTECARIO, mientras que REQ-F-007/REQ-F-008/REQ-F-016 (y HU-01/HU-02/HU-F04) describen a BIBLIOTECARIO como actor principal de ambas operaciones. Confirmado con Marlon que era un bug, no una decisión de diseño; se corrigió el `@PreAuthorize` de ambos métodos a `hasAnyRole('BIBLIOTECARIO','GERENTE','ADMIN')`, verificado con tests reales (`PrestamoControllerSecurityTest`, `crear_conRolBibliotecario_sePermite/ registrarDevolucion_conRolBibliotecario_sePermite`, más `crear_conRolLector_seRechaza/ registrarDevolucion_conRolLector_seRechaza`, confirmando que LECTOR sigue sin acceso). La ruta simple y `DevolucionController` quedan hoy con el mismo conjunto de roles permitidos para devolución.
3. **Confirma, con evidencia adicional, la asimetría que REQ-NF-010 ya reconocía** (antes solo documentada para `LibroController` vs `Prestamo/ReservacionController` respecto a ADMIN): `PrestamoController` (listado por usuario/activos) y `ReservacionController` (crear, listar por usuario) excluyen explícitamente ADMIN de operaciones de lectura/ creación que sí tiene disponibles en `LibroController`. Tras construir esta tabla completa, la asimetría **no** resulta ser una decisión de negocio documentada en ningún ADR — se mantiene como hueco real heredado, ahora con el inventario completo de dónde ocurre.

A24 — Catálogo de estados y transiciones (referencia) El catálogo cerrado de estados de las 5 entidades del dominio (usuarios, libros, prestamos, multas, reservaciones) y la tabla de transiciones reales (evento/actor que dispara cada cambio, con las notas de honestidad sobre VENCIDO, EN_REPARACION/PERDIDO y el estado PENDIENTE de `estados_libro` referenciado en código pero ausente del seed) ya se documentaron en la **sección 1.3.1** de este mismo documento (Bloque 2, M12, de esta actualización). Esta entrada existe como referencia formal desde la lista de anexos, sin duplicar ese contenido — ver sección 1.3.1 para el detalle completo.

4. Trazabilidad

La trazabilidad completa de los 71 requisitos hacia historia de usuario, caso de uso, módulo/endpoint, prueba automatizada, tipo de acceso a datos, evidencia empírica y estado vive en **docs/trazabilidad/matriz.csv**, validada

automáticamente en cada ejecución de CI por `scripts/validate-traceability.sh` (el commit que integró este script a CI ya no es citable por hash puntual, invalidado por la reescritura de historia con `git-filter-repo` señalada en la portada de este documento; el script está confirmado presente y funcional en el tag `v1.0.0`, commit `16279881`, y extendido en esta misma revisión — ver `M23/CHANGELOG-REQ.md`). Este SRS no reemplaza esa matriz — la expande en prosa formal IEEE 29148 (rationale, criterio de aceptación medible, método de verificación explícito) mientras la matriz sigue siendo la fuente machine-readable para validación automática. Si un requisito nuevo se agrega al sistema a futuro, el proceso correcto es: (1) agregar la fila a `matriz.csv`, (2) expandir la entrada correspondiente en este SRS, en ese orden — nunca solo uno de los dos, por el mismo riesgo de desincronización ya documentado en ADR-013 para `Flyway/schema.sql`.

4.1 Estados de verificación: implementado vs verificado

La columna `estado` de la matriz usa un vocabulario cerrado de tres valores (`pendiente` / `implementado` / `verificado`, impuesto por la validación 5 de `scripts/validate-traceability.sh`), con la siguiente diferencia operacional entre los dos estados no pendientes:

- **implementado**: el código real existe en el repositorio y hay una evidencia empírica declarada (verificación manual, capturas, inspección de código o configuración, informes de medición citados en `evidencia_empirica`), pero no hay una prueba automatizada que lo re-verifique en cada build.
- **verificado**: además de lo anterior, la fila tiene `prueba_automatizada` no vacía en la matriz — que es literalmente lo que ya impone la validación 2 del script. Solo este estado se re-comprueba en cada ejecución de CI.

Conteo real sobre `matriz.csv` (generado con `csv.DictReader`, no copiado a mano): **23 verificado / 47 implementado / 1 pendiente sobre 71 filas**; de los 26 requisitos Must, 23 están verificado y 3 implementado. La mayoría de los requisitos nuevos del Bloque 5 (REQ-F-032-042, REQ-NF-016-024) quedó en `implementado` porque se redactaron a partir de funcionalidad ya implementada en el código y verificada por inspección al momento de redactar el requisito; automatizar una prueba específica para cada uno es trabajo pendiente que no bloqueó esta entrega.

5. Requisitos de calidad de software (ISO/IEC 25010)

`docs/arquitectura/ISO25010.md` ya mapea las 8 características de calidad de producto de ISO/IEC 25010 contra escenarios concretos de SGB-SaaS y la estrategia que atiende cada una, con prioridad asignada por criterio del equipo (no medición, salvo donde se cita evidencia real). Este SRS no duplica esa tabla completa; la resume aquí para dejar explícita la relación con los requisitos no funcionales de la sección 3.2:

Característica ISO 25010	Prioridad	Requisito(s) NF relacionado(s)
Adecuación funcional	Alta	REQ-F-007, REQ-F-008, REQ-NF-004, REQ-F-020 (verificación de correo, ahora parte del flujo obligatorio de alta de cuenta)
Eficiencia de desempeño	Media	REQ-NF-003; prueba de carga formal (k6, 5 corridas, comparación estadística Wilcoxon/Cliff's delta) en <code>docs/mediciones/perf/REPORT.md</code>
Compatibilidad	Media	3.3 (interfaz REST/JSON)
Usabilidad	Alta	REQ-F-016, REQ-F-013 (mensajes explícitos en UI); evidencia empírica SUS todavía pendiente (OBS-08)
Fiabilidad	Alta	REQ-NF-001 (riesgo fail-open/fail-closed de Redis, ver A15 para el detalle por servicio: <code>JwtAuthFilter/VerificacionCorreoService</code> fail-closed, <code>LoginRateLimiter/ChatbotRateLimiter</code> fail-open); mismo riesgo se extiende a <code>ChatbotRateLimiter</code> (REQ-F-028) y <code>VerificacionCorreoService</code> (REQ-F-020), ambos también respaldados por Redis
Seguridad	Alta	REQ-NF-001, 002, 006, 007, 010, 011, 012 (implementado en producción real — ver nota abajo), 013, 014a-d (los 4 sub-requisitos implementados — ver nota abajo); REQ-F-028 (manejo de la API key de Gemini: nunca se registra en logs la URL que la contiene, ver <code>GeminiClient</code> /ADR-016 y su análisis de qué datos se envían al proveedor externo)
Mantenibilidad	Alta	14 ADRs de <code>docs/adr/</code> (cifra recontada en este commit, 2026-09-07 — incluye <code>adr-029-v29-gap.md</code> , agregado después de la versión anterior de este SRS que citaba 13), <code>docs/basedatos/CATALOGO-SP.md</code> , REQ-NF-015 (CI/CD, <code>Makefile</code>)
Portabilidad	Alta	REQ-NF-005, REQ-NF-009

Nota sobre REQ-NF-012/REQ-NF-014a-d (actualizada respecto a versiones anteriores de este SRS, que los marcaban como completamente pendientes o parcialmente pendientes; REQ-NF-014 se dividió en REQ-NF-014a-d en esta misma revisión, ver Bloque 4/M14a): esta revisión (hallazgo del Dr. Guerrero) confirma que todos están **implementados** en lo que a este SRS le corresponde declarar — REQ-NF-012 en el despliegue real de producción (Render termina TLS en su borde; el stack Docker Compose local, fuera del alcance de este requisito, sigue en HTTP plano) y REQ-NF-014a-d (CSP backend+frontend, supresión de stacktraces, Swagger desactivado en producción, contenedor sin root) — ver el detalle verificado en cada requisito, sección 3.2.2.

6. Notas de honestidad y gaps conocidos (resumen)

Consolidado de todas las notas de honestidad ya señaladas en línea en la sección 3, para que quien audite este documento no tenga que buscarlas una por una:

1. **REQ-F-001**: solo 1 de 3 criterios de aceptación tiene prueba automatizada de regresión (rechazo por correo duplicado); el flujo exitoso y el rechazo por contraseña corta no tienen test.
2. **REQ-F-010**: sin HU/CU que documente su motivación de negocio; el rationale de este SRS es una inferencia razonable, no un hecho documentado previamente.
3. **REQ-F-012**: sin HU dedicada; se infiere de la implementación (mismo componente que REQ-F-011).
4. **REQ-NF-002**: el `accessToken` sigue sin migrar a cookie `HttpOnly` (solo el `refreshToken` lo está) — gap real y deliberadamente diferido, no un olvido.
5. **REQ-NF-010**: asimetría real de roles entre `LibroController` (incluye ADMIN) y `PrestamoController/ReservacionContr` (no lo incluyen).
6. **REQ-NF-012 y REQ-NF-014a-d**: versiones anteriores de este SRS los declaraban primero **pendientes** y luego **parcialmente implementados** (TLS real end-to-end y CSP del `nginx.conf` seguían sin cerrar; REQ-NF-014 era un único requisito compuesto, dividido en esta revisión en REQ-NF-014a-d, ver M14a). Esta revisión (hallazgo del Dr. Guerrero, quien pidió distinguir el despliegue Docker local del despliegue real) verifica todos contra el estado real: REQ-NF-012 **sí** corre bajo HTTPS real en producción (Render termina TLS en su borde para `sgb-backend/biblora-sgb`, `render.yaml` sin dominio propio); REQ-NF-014a **sí** tiene CSP en `frontend-angular/nginx.conf:10` además del backend. Todos se declaran implementados en lo que corresponde a este sistema; lo que sigue sin TLS/certificado propio es exclusivamente el stack de Docker Compose local, que nunca fue el objetivo de estos requisitos.
7. **REQ-NF-013**: verificado por inspección manual puntual durante la auditoría original, sin test de regresión permanente en el suite.
8. **HU/CU de Cajas (HU-01 a HU-05, CU-01 a CU-05)**: viven consolidadas en `docs/requisitos/historias-usuario.m` y `docs/requisitos/casos-de-uso.md`, no como un archivo por HU/CU como el resto de módulos (`docs/requisitos/historias/`, `docs/requisitos/casos-de-uso/`) — inconsistencia real de convención de archivos entre módulos, documentada aquí en vez de normalizada silenciosamente (normalizarla sería una tarea de refactor de documentación fuera del alcance de este SRS).
9. **ADRs**: la versión anterior de este SRS (Tercera Entrega) señalaba que el resumen ejecutivo de `docs/informe-entrega-3.tex` corregía una cifra de “13 ADRs” a los 10 reales existentes en `docs/adr/` en ese momento. Tras agregar ADR-014/015/016 tres módulos después, `docs/adr/` llegó a tener 13 ADRs reales — la misma cifra que en su momento era incorrecta, coincidencia ya señalada por versiones previas de este SRS. **Recuento de esta revisión (hallazgo del Dr. Guerrero, 2026-09-07)**: `docs/adr/` tiene hoy **14 ADRs reales** (recontado con `ls docs/adr/` excluyendo `README.md`) — se agregó `adr-029-v29-gap.md` (numeración de migraciones Flyway, hueco V29) después de la última vez que este SRS contó 13. Este SRS usa la cifra verificada en este commit (14), sin asumir que el número anterior seguía vigente.
10. **Diagrama de clases UML**: `docs/observaciones/OBSERVACIONES.md` (OBS-02) ya documenta que este diagrama sigue sin versionar como imagen en el repositorio — no es un gap de este SRS, es un gap heredado y ya reportado en su propia bitácora de observaciones.
11. **REQ-F-017, REQ-F-018, REQ-F-023, REQ-F-024**: la matriz cita HU/CU (HU-CFG-01/CU-CFG-01, HU-PRE-03/CU-07, HU-ADM-01/CU-ADM-01, HU-AUD-01/CU-AUD-01) que **no existen** como archivo en `docs/requisitos/historias/` ni `docs/requisitos/casos-de-uso/` — verificado por búsqueda exhaustiva en todo `docs/requisitos/` antes de escribir esta versión del SRS, no asumido. El rationale y los criterios de aceptación de estos 4 requisitos se redactaron a partir del código real (`controllers/services/tests`), nunca inventando el contenido de una HU/CU Gherkin que no existe.
12. **REQ-F-019**: la matriz cita HU-PRE-04 (tampoco existe como archivo, mismo gap que el punto anterior) pero también CU-01, que **sí existe** — es el mismo caso de uso “Registrar préstamo” que ya respalda REQ-F-007. Se documenta como una reutilización deliberada (el QR es un mecanismo alternativo de identificación para la misma acción de negocio), no como un error de la matriz asumido sin verificar.

13. **REQ-F-020 vs. REQ-F-001:** el estado inicial de una cuenta tras el registro cambió de **ACTIVO** (como documentaba REQ-F-001 hasta la versión anterior de este SRS) a **PENDIENTE_VERIFICACION**. Esta versión **corrige REQ-F-001** para eliminar esa contradicción interna (hallazgo del Dr. Guerrero, auditoría ISO/IEC/IEEE 29148:2018 sobre el tag v1.0.0) — ver el campo “Depende de: REQ-F-020” agregado a REQ-F-001 y docs/requisitos/CHANGELOG-REQ.md para el detalle del cambio.
 14. **Sección 3.3 (interfaz externa):** la cifra de endpoints/controllers se actualizó (19→44 endpoints, 5→15 controllers) contando directamente sobre el código de este commit; docs/informe-entrega-3.tex y docs/postman/coleccion.json (que sigue citando 39 requests) **no** se actualizaron como parte de esta tarea — quedan fuera de su alcance, con el mismo tipo de desactualización que este SRS tenía antes de esta versión.
 15. **REQ-F-025, REQ-F-026, REQ-F-027, REQ-F-021, REQ-F-022a/b/c, REQ-F-028:** sin HU/CU en absoluto (la propia matriz los marca con - en ambas columnas, no una omisión de este SRS) — mismo criterio que REQ-F-010 ya establecía en la versión anterior.
-

Aprobación del docente-director

El presente documento (SRS v1.0.0) se somete a revisión y aprobación del docente-director del Proyecto Fin de Curso.

Docente-director: Dr. Gleiston Cicerón Guerrero Ulloa, Ph.D.

Firma: _____